

CUTILE PROGRAMMING

Bryce Adelstein Lelbach

Principal Architect

✉ brycelelbach@gmail.com [@blelbach](https://twitter.com/blelbach)



Grid

Easy acceleration of
bulk data operations



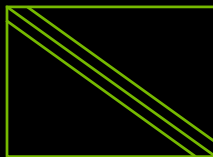
System splits work
into blocks, divides
data into tiles, & maps
both onto threads

```
import cupy as cp

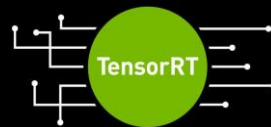
samples = ...
sort     = cp.sort(samples)
cdf      = cp.cumsum(sort) / sort.sum()
p95      = sort[cp.searchsorted(cdf, 0.95)]
```



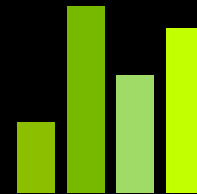
cuBLAS



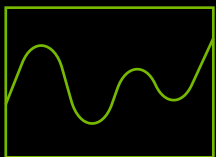
cuSPARSE



**cuFile
(GDS)**



NVBench



cuFFT

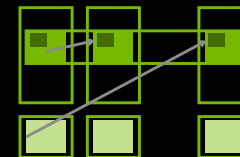


cuDSS

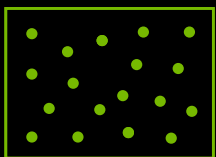
**CV
CUDA**

cuLitho

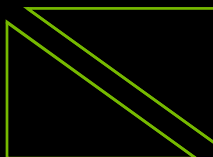
**Cooperative
Groups**



NVSHMEM



cuRAND

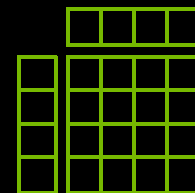


cuSOLVER

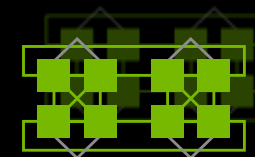
RAPIDS



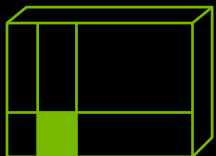
CUDA-Q



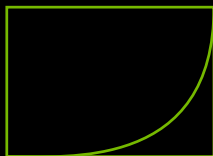
CUTLASS



NCCL



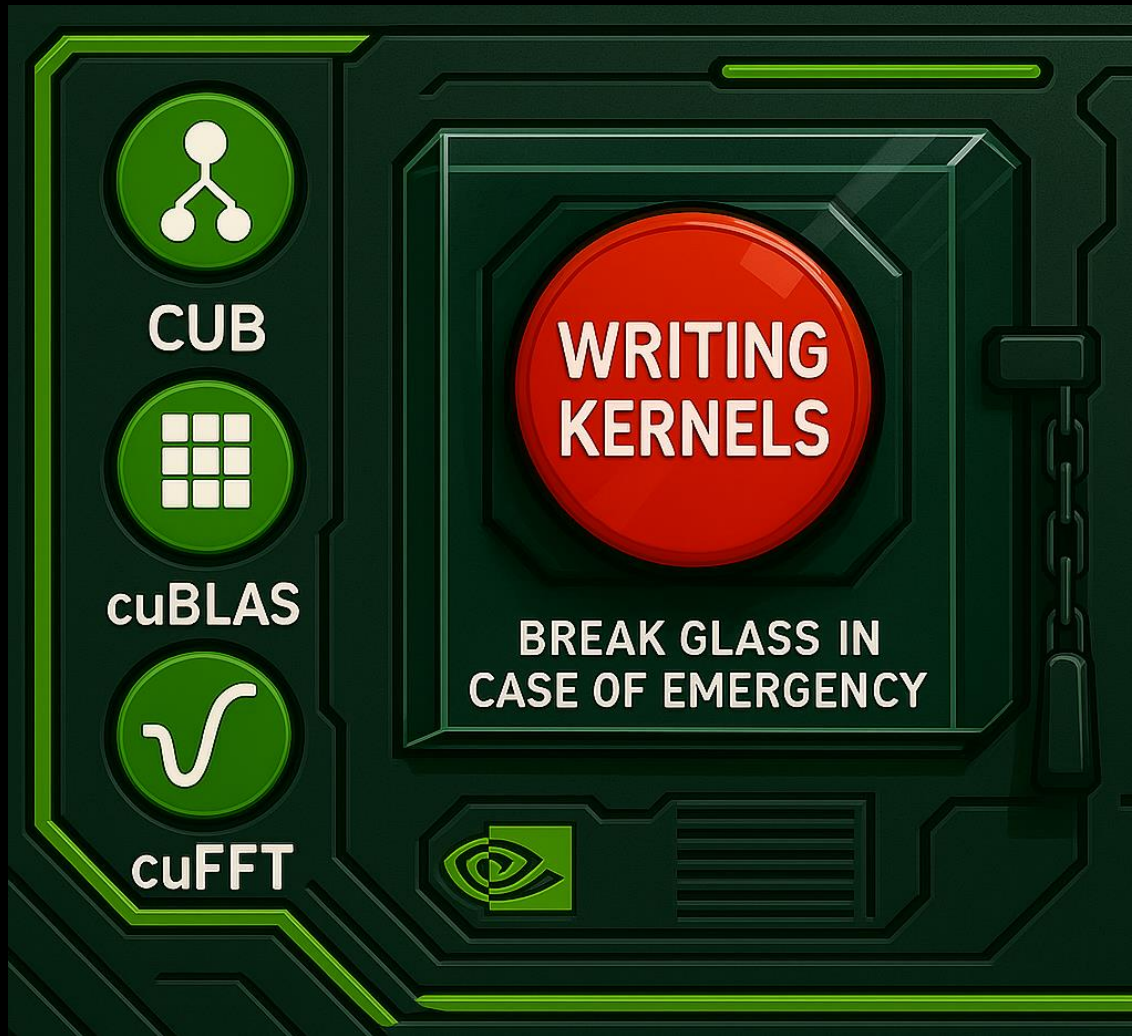
cuTENSOR



CUDA Math



**CUDA
Runtime**



Prefer using libraries!

If no library exists or
you can't get the perf
you need with them,
then...

```

@cuda.device.kernel
def matmul(A, B, C):
    TPB = cuda.blockDim.x
    BPG = cuda.gridDim.x
    x, y = cuda.grid(2)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    sA = cuda.shared.array((TPB, TPB), float32)
    sB = cuda.shared.array((TPB, TPB), float32)

    tmp = float32(0.)
    for i in range(BPG):
        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx+i*TPB) < A.shape[1]:
            sA[ty, tx] = A[y, tx + i * TPB]
        if x < B.shape[1] and (ty+i*TPB) < B.shape[0]:
            sB[ty, tx] = B[ty + i * TPB, x]

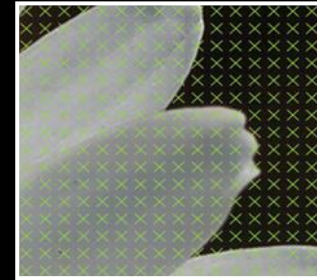
        cuda.syncthreads()
        for j in range(TPB):
            tmp += sA[ty, j] * sB[j, tx]
        cuda.syncthreads()

    if y < C.shape[0] and x < C.shape[1]:
        C[y, x] = tmp

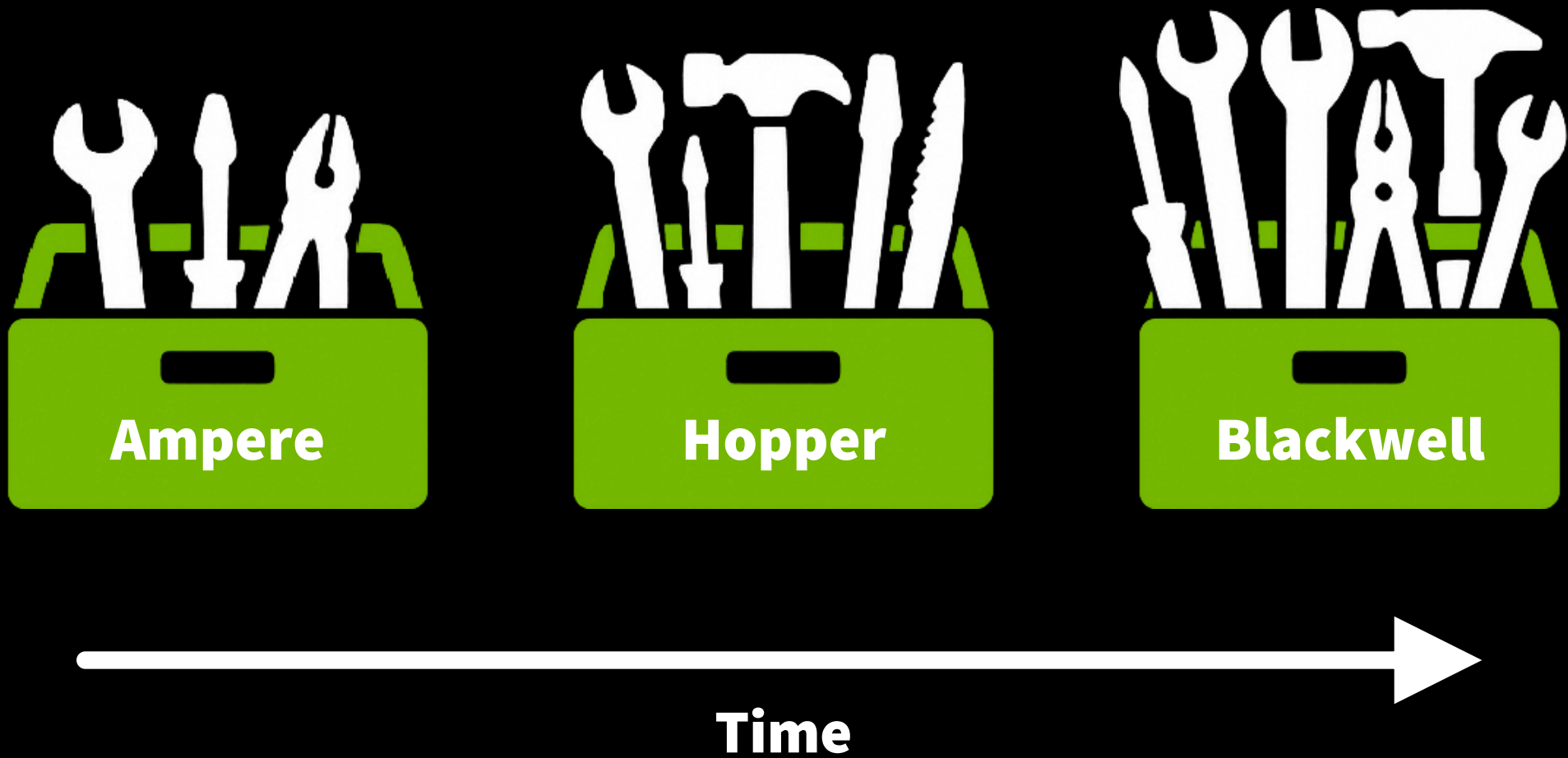
```

SIMT

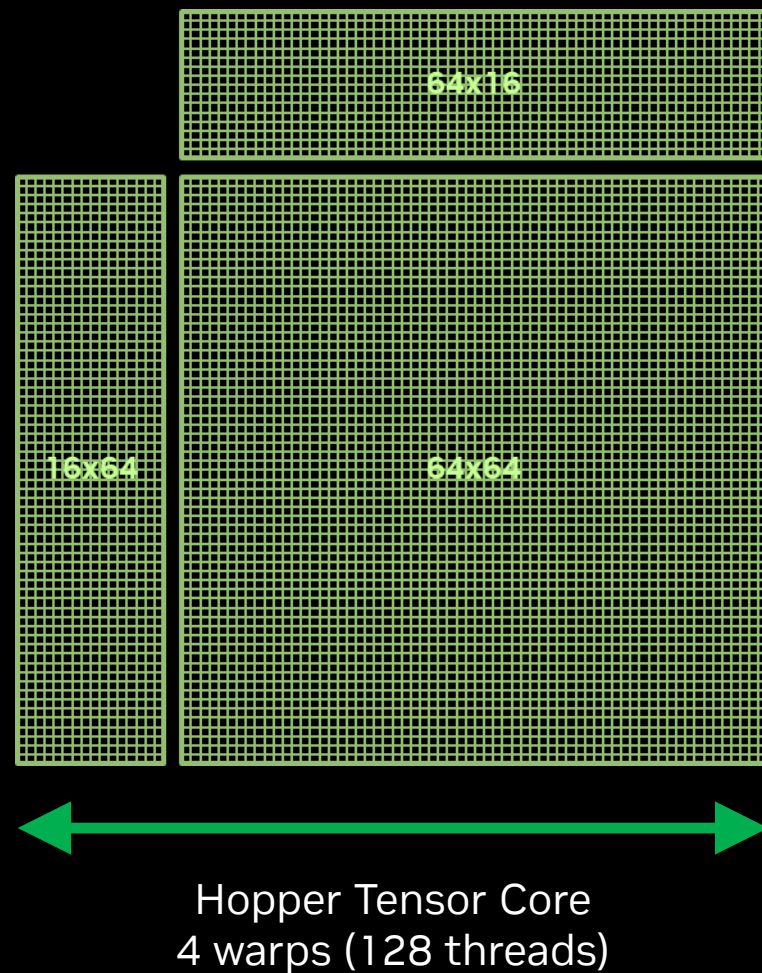
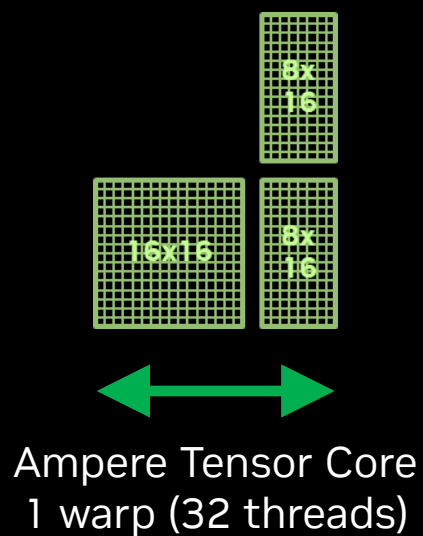
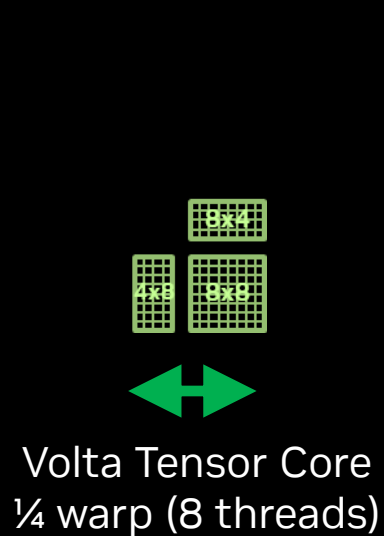
Full control for
all parallel workloads



User splits work into
blocks, divides data into
tiles, & maps both onto
threads



Tensor core structure (formats, precisions, width, mapping to warps, etc) have changed with every GPU architecture, forcing us to rewrite our code!



Grid

Easy acceleration of bulk data operations



System splits work into blocks, divides data into tiles, & maps both onto threads

Tile

Productive, portable, & performant array ops

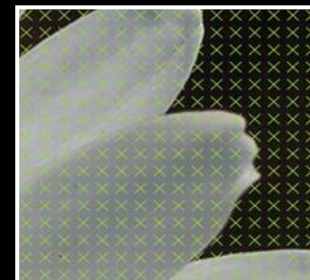


User splits work into blocks & divides data into tiles

System maps both onto threads

SIMT

Full control for all parallel workloads



User splits work into blocks, divides data into tiles, & maps both onto threads

cuTile: Tile Programming for GPUs

Array-oriented & easy to use.

Performant & portable across CUDA.

```
import cuda.tile as ct

@ct.kernel
def softmax(I, O, r: ct.CInt, c: ct.Cint):
    T = ct.load(I, (ct.bid(0), 0), (r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, (ct.bid(0), 0), smax)
```

cuTile: Tile Programming for GPUs

Array-oriented & easy to use.
Performant & portable across CUDA.

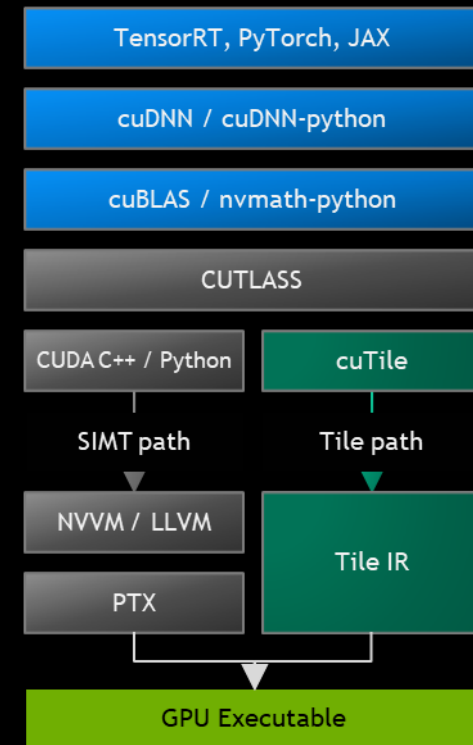
```
import cuda.tile as ct

@ct.kernel
def softmax(I, O, r: ct.CInt, c: ct.Cint):
    T = ct.load(I, (ct.bid(0), 0), (r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, (ct.bid(0), 0), smax)
```

Tile IR: A New Low-Level Compiler For Tile Programs



Naïve CUDA Python SIMT

```
@cuda.device.kernel
def matmul(A, B, C):
    TPB = cuda.blockDim.x
    BPG = cuda.gridDim.x
    x, y = cuda.grid(2)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    sA = cuda.shared.array((TPB, TPB), float32)
    sB = cuda.shared.array((TPB, TPB), float32)

    tmp = float32(0.)
    for i in range(BPG):
        sA[ty, tx] = 0
        sB[ty, tx] = 0
        if y < A.shape[0] and (tx+i*TPB) < A.shape[1]:
            sA[ty, tx] = A[y, tx + i * TPB]
        if x < B.shape[1] and (ty+i*TPB) < B.shape[0]:
            sB[ty, tx] = B[ty + i * TPB, x]

        cuda.syncthreads()
        for j in range(TPB):
            tmp += sA[ty, j] * sB[j, tx]
        cuda.syncthreads()

    if y < C.shape[0] and x < C.shape[1]:
        C[y, x] = tmp
```

cuTile Python

```
import cuda.tile as ct

@ct.kernel
def matmul(X, Y, Z,
           mt: ct.Constant[Int],
           nt: ct.Constant[int],
           nk: ct.Constant[Int]):
    Zt = ct.zeros((mt, nt), C.dtype)

    (m, n) = (X.shape[0], Y.shape[1])
    (i, j) = ct.swizzle2d(m, n, mt, nt)

    nk = ct.num_tiles(X, 1, (mt, kt))
    for k in range(nk):
        Xt = ct.load(X, (i, k), (mt, kt))
        Yt = ct.load(Y, (k, j), (kt, nt))
        Zt = ct.mma(Xt, Yt, Zt)

    ct.store(Z, (i, j), Zt)
```

Speed-Of-Light Non-Portable CUDA C++ SIMT

```
using namespace cute;

template <class SharedStorage,
          class ATensor, class CTensor, class DTensor,
          class MmaTiler_MNK, class TiledMMA, class ClusterShape_MNK,
          class TmaAtomA, class TmaAtomB,
          class Alpha, class Beta>
__global__ static
void
matmul(ATensor mA,
       BTensor mB,
       CTensor mC,
       DTensor mD,
       MmaTiler_MNK mma_tiler,
       TiledMMA tiled_mma,
       ClusterShape_MNK cluster_shape,
       CUTE_GRID_CONSTANT TmaAtomA const tma_atom_A,
       CUTE_GRID_CONSTANT TmaAtomB const tma_atom_B,
       Alpha alpha, Beta beta)
{
    layout cluster_layout_vmnk = tiled_divide(make_layout(cluster_shape),
        make_tile(typename TiledMMA::AtomThrID()));
    auto mma_coord_vmnk = make_coord(blockIdx.x % size<0>(cluster_layout_vmnk),
        blockIdx.x / size<0>(cluster_layout_vmnk),
        blockIdx.y,
        ...);

    auto mma_coord = select<1,2,3>(mma_coord_vmnk);
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<1, X, 1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step<X, 1, 1>{});
    Tensor gC = local_tile(mC, mma_tiler, mma_coord, Step<1, 1, X>{});
    Tensor gD = local_tile(mD, mma_tiler, mma_coord, Step<1, 1, X>{});
    __syncthreads();

    extern __shared__ char shared_memory[];
    SharedStorage& shared_storage =
        *reinterpret_cast<SharedStorage*>(shared_memory);

    Tensor tCsA = shared_storage.tensor_sA();
    Tensor tCsB = shared_storage.tensor_sB();

    auto mma_v = get<0>(mma_coord_vmnk);
    ThrMMA cta_mma = tiled_mma.get_slice(mma_v);
    Tensor tCgA = cta_mma.partition_A(gA);
    Tensor tCgB = cta_mma.partition_B(gB);
    Tensor tCgC = cta_mma.partition_C(gC);
    Tensor tCgD = cta_mma.partition_D(gD);
    __syncthreads();

    Tensor tCrA = cta_mma.make_fragment_A(tCsA);
    Tensor tCrB = cta_mma.make_fragment_B(tCsB);

    Tensor tCtAcc = cta_mma.make_fragment_C(tCgC);

    uint32_t elect_one_thr = cute::elect_one_sync();
    uint32_t elect_one_warp = (threadIdx.x / 32 == 0);

    using TmemAllocator = cute::TMEM::Allocator1Sm;
    TmemAllocator tmem_allocator{};

    if (elect_one_warp) {
        tmem_allocator.allocate(TmemAllocator::Sm100TmemCapacityColumns,
            &shared_storage.tmem_base_ptr);
    }

    __syncthreads();
    tCtAcc.data() = shared_storage.tmem_base_ptr;
    __syncthreads();

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, Int<0>{}, Layout<_1>{},
        group_modes<0,3>(tCsA), group_modes<0,3>(tCgA));
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, Int<0>{}, Layout<_1>{},
        group_modes<0,3>(tCsB), group_modes<0,3>(tCgB));
    __syncthreads();

    if (elect_one_warp && elect_one_thr) {
        cute::initialize_barrier(shared_storage.mma_barrier, /* num_ctas */ 1);
        cute::initialize_barrier(shared_storage.tma_barrier, /* num_threads */ 1);
    }

    int mma_barrier_phase_bit = 0;
    int tma_barrier_phase_bit = 0;
    __syncthreads();

    tiled_mma.accumulate_ = UMMMA::ScaleOut::Zero;
    for (int k_tile = 0; k_tile < size<3>(tCgA); ++k_tile)
    {
        if (elect_one_warp && elect_one_thr) {
            int tma_transaction_bytes = sizeof(make_tensor_like(tAsA))
                + sizeof(make_tensor_like(tBsB));
            cute::set_barrier_transaction_bytes(shared_storage.tma_barrier,
                tma_transaction_bytes);
            copy(tma_atom_A.with(shared_storage.tma_barrier), tAgA(_k_tile), tAsA);
            copy(tma_atom_B.with(shared_storage.tma_barrier), tBgB(_k_tile), tBsB);
        }

        cute::wait_barrier(shared_storage.tma_barrier, tma_barrier_phase_bit);
        tma_barrier_phase_bit ^= 1;

        if (elect_one_warp) {
            for (int k_block = 0; k_block < size<2>(tCrA); ++k_block) {
                gemm(tiled_mma, tCrA(_k_block), tCrB(_k_block), tCtAcc);
                tiled_mma.accumulate_ = UMMMA::ScaleOut::One;
            }
            cutlass::arch::umma_arrive(&shared_storage.mma_barrier);
        }
        cute::wait_barrier(shared_storage.mma_barrier, mma_barrier_phase_bit);
        mma_barrier_phase_bit ^= 1;

        TiledCopy tiled_t2r_copy = make_tmem_copy(SM100_TMEM_LOAD_32dp32b1x(), tCtAcc);
        ThrCopy thr_t2r_copy = tiled_t2r_copy.get_slice(threadIdx.x);

        Tensor tDgC = thr_t2r_copy.partition_D(tCgC);
        Tensor tDrC = make_fragment_like(tDgC);
        copy(tDgC, tDrC);

        Tensor tDtAcc = thr_t2r_copy.partition_S(tCtAcc);
        Tensor tDgD = thr_t2r_copy.partition_D(tCgD);

        using AccType = typename decltype(tCtAcc)::value_type;
        Tensor tDrAcc = make_tensor<AccType>(shape(tDgD));

        copy(tiled_t2r_copy, tDtAcc, tDrAcc);

        axpby(alpha, tDrAcc, beta, tDrC);
        copy(tDrC, tDgD);

        __syncthreads();

        if (elect_one_warp) {
            tmem_allocator.release_allocation_lock();
            tmem_allocator.free(shared_storage.tmem_base_ptr,
                TmemAllocator::Sm100TmemCapacityColumns);
        }
    }
}
```

cuTile Python

```
import cuda.tile as ct
```

```
@ct.kernel
def matmul(X, Y, Z,
           mt: ct.Constant[Int],
           nt: ct.Constant[int],
           nk: ct.Constant[Int]):
    Zt = ct.zeros((mt, nt), C.dtype)
```

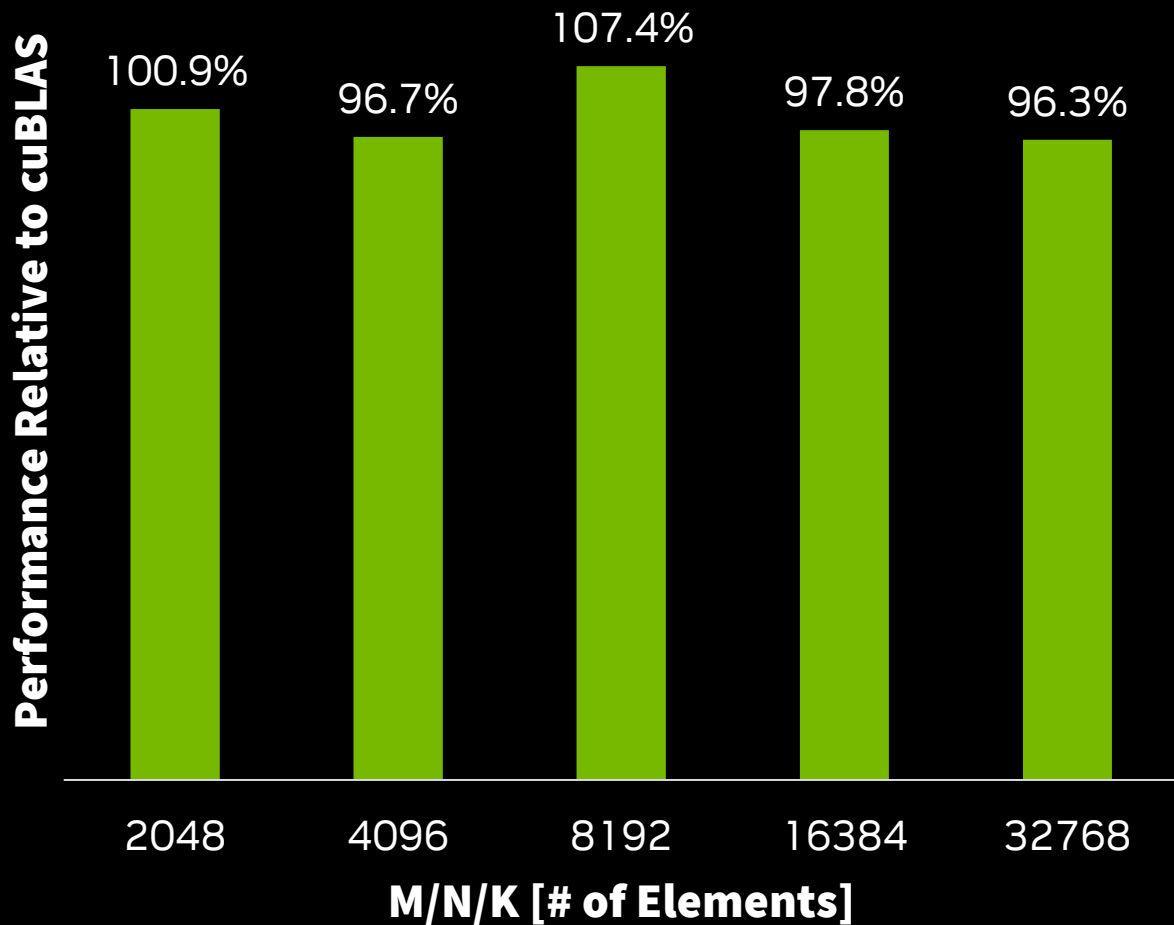
```
(m, n) = (X.shape[0], Y.shape[1])
(i, j) = ct.swizzle2d(m, n, mt, nt)
```

```
nk = ct.num_tiles(X, 1, (mt, kt))
for k in range(nk):
    Xt = ct.load(X, (i, k), (mt, kt))
    Yt = ct.load(Y, (k, j), (kt, nt))
    Zt = ct.mma(Xt, Yt, Zt)
```

```
ct.store(Z, (i, j), Zt)
```

cuTile Matmul Performance

Relative to cuBLAS, FP16, B300, CUDA 13.2



cuTile Python

```
import cuda.tile as ct

@ct.kernel
def matmul(X, Y, Z,
           mt: ct.Constant[Int],
           nt: ct.Constant[int],
           nk: ct.Constant[Int]):
    Zt = ct.zeros((mt, nt), C.dtype)

    (m, n) = (X.shape[0], Y.shape[1])
    (i, j) = ct.swizzle2d(m, n, mt, nt)

    nk = ct.num_tiles(X, 1, (mt, kt))
    for k in range(nk):
        Xt = ct.load(X, (i, k), (mt, kt))
        Yt = ct.load(Y, (k, j), (kt, nt))
        Zt = ct.mma(Xt, Yt, Zt)

    ct.store(Z, (i, j), Zt)
```



```

@ct.kernel(occupancy=2)
def tq_attn_vfused_3bit(
    Q, K_mse, Signs, R_norms, Q_proj,
    V_Indices, V_Norms, Pi, Out,
    scale, corr_scale, seq_k,
    # ...8 centroid values unpacked as constants...
    USE_SWIZZLE: ConstBool,
):
    qb = ct.bid(0)

    # Load Q tile once
    q = ct.load(Q, index=(qb, 0), shape=(BLOCK_Q, HEAD_DIM))

    # Load rotation matrix Pi into shared memory
    pi = ct.load(Pi, index=(0, 0), shape=(HEAD_DIM, HEAD_DIM))

    for kb in range(num_kv_blocks):
        # Score: Q @ K_mse^T
        k = ct.load(K_mse, index=(kb, 0), ...)
        t1 = ct.mma(q, k, transpose_b=True)

        # QJL correction: Q_proj @ Signs^T
        s = ct.load(Signs, index=(kb, 0), ...)
        t2 = ct.mma(qp, s, transpose_b=True)

        # Combine + online softmax
        score = t1 * scale + norms * corr_scale * t2
        m = ct.max(m_prev, ct.max(score))
        p = ct.exp2((score-m)*INV_LOG_2)
        acc = acc*ct.exp2((m_prev-m)*INV_LOG_2) + ct.mma(p,v)
        ...

```

TurboQuant cuTile

*Production-quality kernels built just 9 days after
the paper's release by undergrad student
Anirudh Bharadwaj Vangara*

5.02x

Compression

144.7

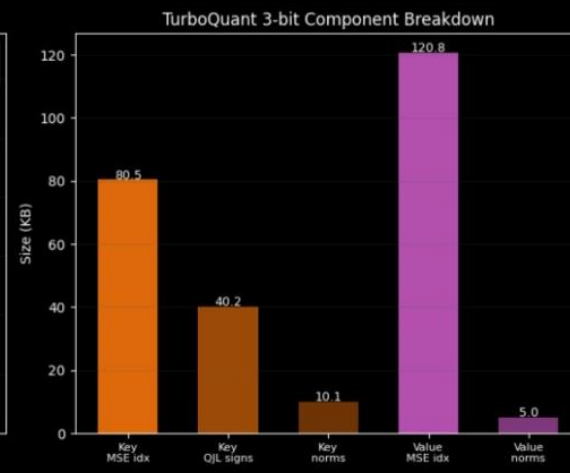
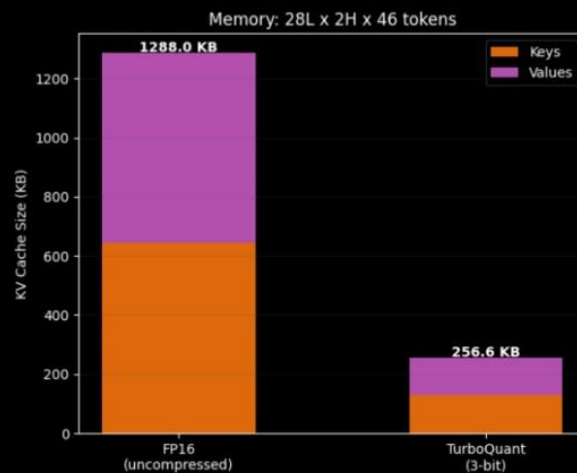
tok/s on B200

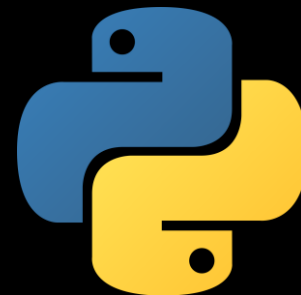
~0.985

Cosine Similarity

5

Kernel Types





```
import cuda.tile as ct

@ct.kernel
def matmul(X: ct.Array, Y: ct.Array, Z: ct.Array,
           mt: ct.Constant[int],
           nt: ct.Constant[int],
           kt: ct.Constant[int]):
    Zt = ct.zeros(shape=(mt, nt), dtype=Z.dtype)

    Xv = X.partition(shape=(mt, kt))
    Yv = Y.partition(shape=(kt, nt))

    (i, j) = ct.swizzle2d(X.shape[0], Y.shape[1], mt, nt)
    for k in range(0, Xv.num_tiles[1]):
        Zt = ct.mma(Xv.load((i, k)), Yv.load((k, j)), Zt)

    ct.store(Z, index=(i, j), tile=Zt)
```

cuTile Python: Available since CUDA 13.1
github.com/nvidia/cutile-python

```
#include <cuda_tile.h>
```

```
template <int mt, int nt, int kt>
__tile__ void matmul(ct::tensor_span_like auto X,
                    ct::tensor_span_like auto Y,
                    ct::tensor_span_like auto Z) {
    using E = ct::element_t<decltype(Z)>;
    auto Zt = ct::zeros<ct::tile<E, ct::shape_v<mt, nt>>>();

    auto Xv = X.partition(ct::shape_v<mt, kt>);
    auto Yv = Y.partition(ct::shape_v<kt, nt>};

    auto [i, j] = ct::swizzle2d<mt, nt>(X.extent(0), Y.extent(1));
    for (auto k : ct::irange(0, Xv.num_tiles(1)))
        Zt = ct::mma(Xv.load(i, k), Yv.load(k, j), Zt);

    Z.partition(ct::shape_v<mt, nt>).store(Zt, i, j);
}
```



cuTile C++: Coming soon in CUDA 13.3!

```
use cutile::core::* as ct;
```

```
#[cutile::entry()]
```

```
fn matmul<E, const mt: i32, const nt: i32, const kt: i32>(
    X: &ct::Tensor<E, {[-1, -1]}>, Y: &ct::Tensor<E, {[-1, -1]}>,
    Z: &mut ct::Tensor<E, {[mt, nt]}>,
```

```
) {
    let mut Zt = ct::zeros(ct::const_shape![mt, nt]);
```

```
    let Xv = X.partition(ct::const_shape![mt, kt]);
```

```
    let Yv = Y.partition(ct::const_shape![kt, nt]);
```

```
    let (i, j, _) = ct::swizzle2d::<mt, nt>(X.shape(0), Y.shape(1));
```

```
    for k in 0i32..Xv.num_tiles(1)
```

```
        { Zt = ct::mma(Xv.load([i, k]), Yv.load([k, j]), Zt); }
```

```
    Z.store(Zt);
```

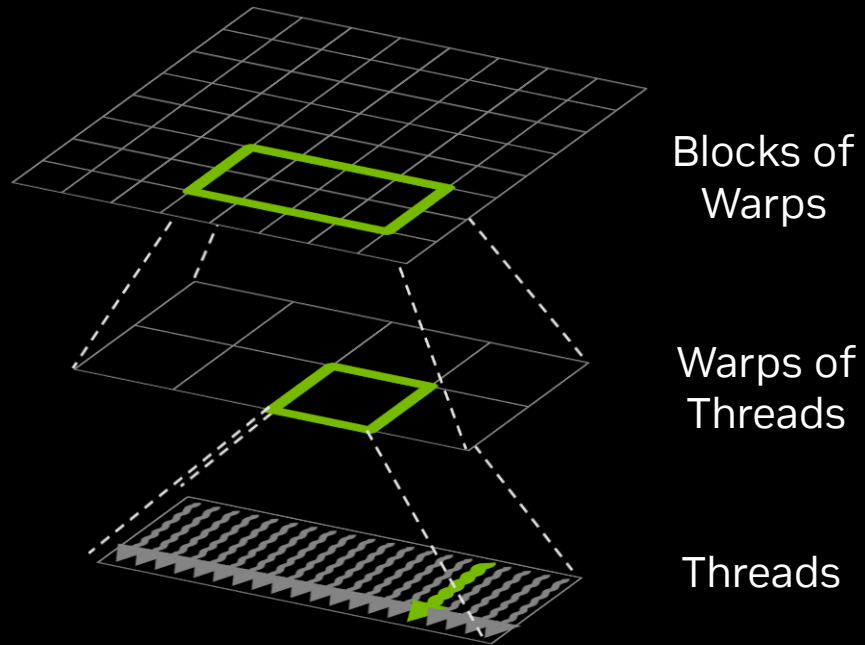
```
}
```



cuTile Rust: Research project available today!
<https://github.com/nvmlabs/cutile-rs>

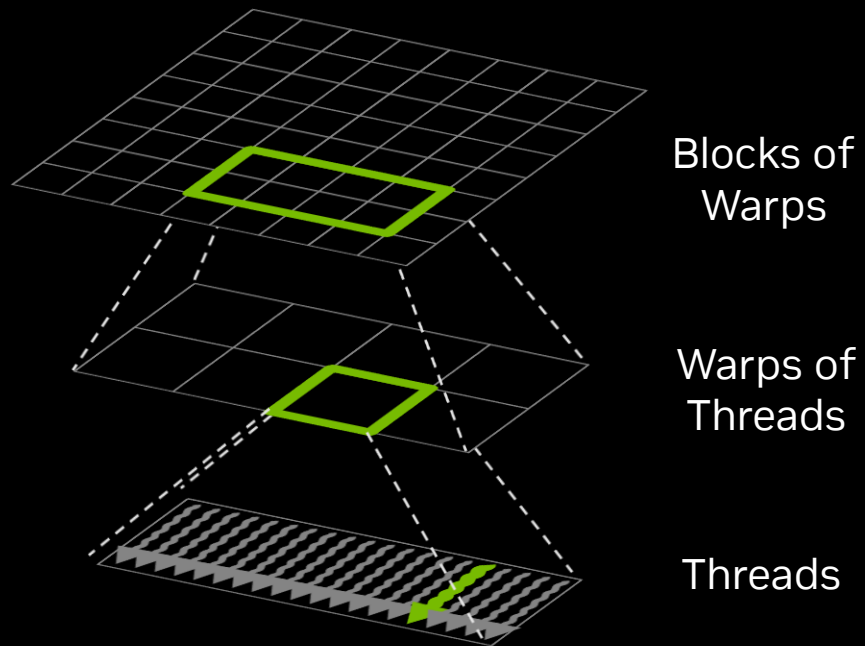
Unit of	SIMT Model
Execution	Thread

Thread Grid

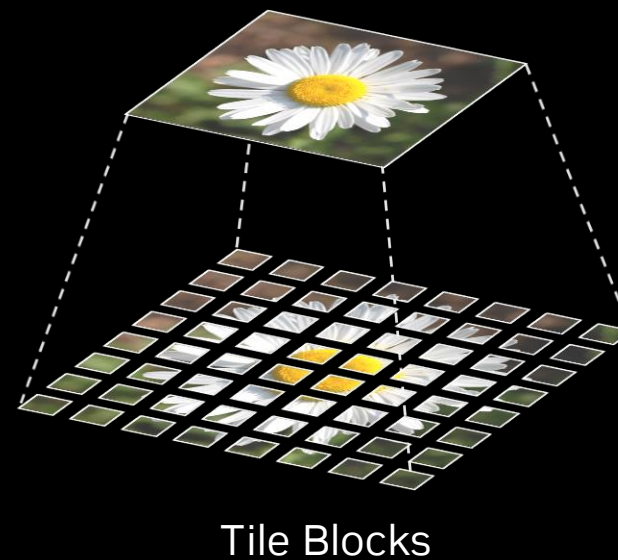


Unit of	SIMT Model	Tile Model
Execution	Thread	Block

Thread Grid



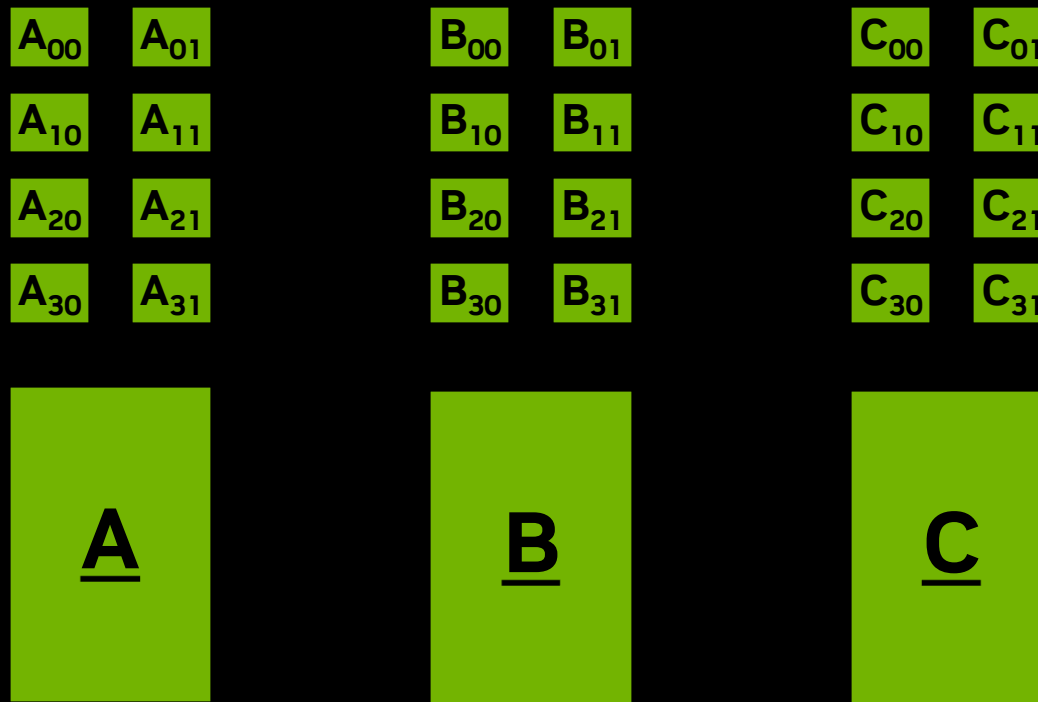
Block Grid



Unit of	SIMT Model	Tile Model
Execution	Thread	Block
Data	Element	

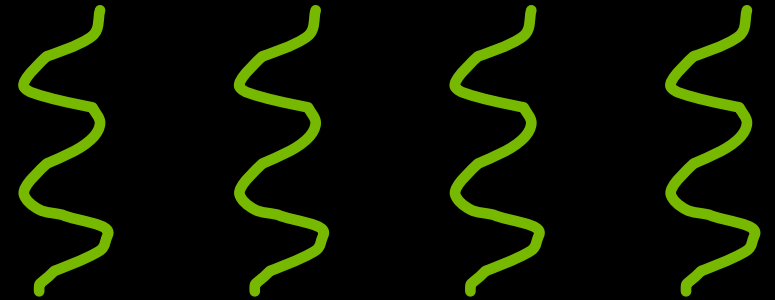
A_{00}	A_{01}		B_{00}	B_{01}		C_{00}	C_{01}
A_{10}	A_{11}		B_{10}	B_{11}		C_{10}	C_{11}
A_{20}	A_{21}		B_{20}	B_{21}		C_{20}	C_{21}
A_{30}	A_{31}		B_{30}	B_{31}		C_{30}	C_{31}

Unit of	SIMT Model	Tile Model
Execution	Thread	Block
Data	Element	Array



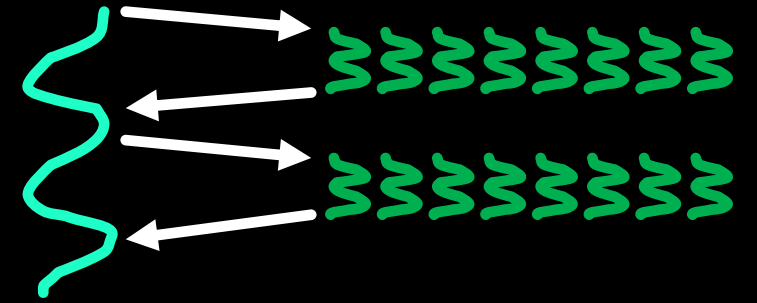
```
@cuda.device.kernel
def vadd(X, Y, Z, tshape):
    bid = cuda.grid(1)
    for i in range(tshape / cuda.blockDim.x):
        item = bid + i * cuda.blockDim.x
        if (item < len(Z))
            Z[item] = X[item] + Y[item]
```

Thread 0 runs vadd Thread 1 runs vadd Thread 2 runs vadd Thread 3 runs vadd



```
@cuda.tile.kernel
def vadd(X, Y, Z, tshape: ct.Constant[int]):
    Xt = cuda.tile.load(X, cuda.tile.bid(0), tshape)
    Yt = cuda.tile.load(Y, cuda.tile.bid(0), tshape)
    Zt = Xt + Yt
    cuda.tile.store(Z, cuda.tile.bid(0), Zt)
```

A single control thread runs vadd Array ops are parallelized across the block's execution resource



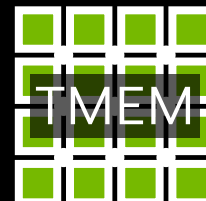
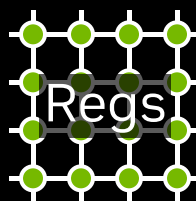
Split work into blocks
Divide data into tiles



Map blocks & tiles
onto threads



Tile storage?

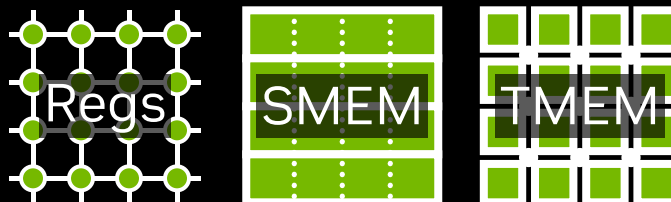




Tile memory layout?

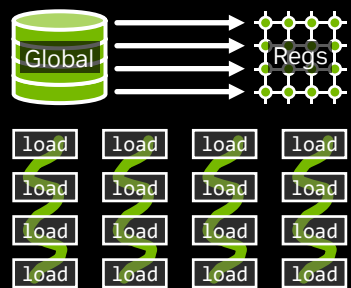


Tile storage?

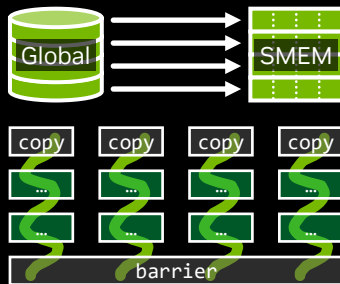


Memory movement strategy?

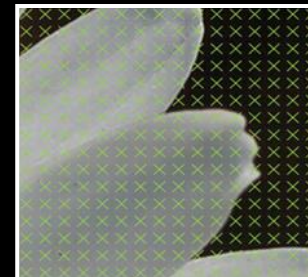
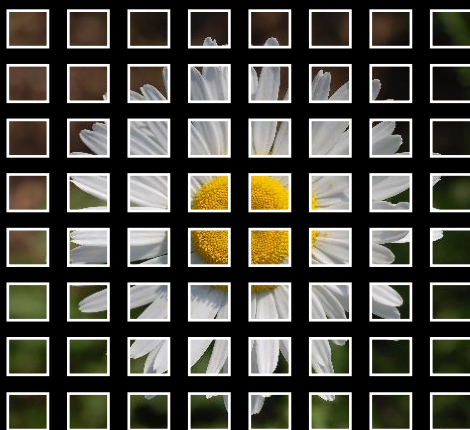
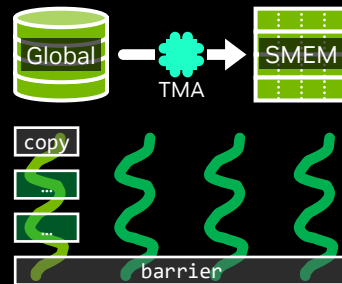
Synchronous Thread Load



Asynchronous Thread Copy



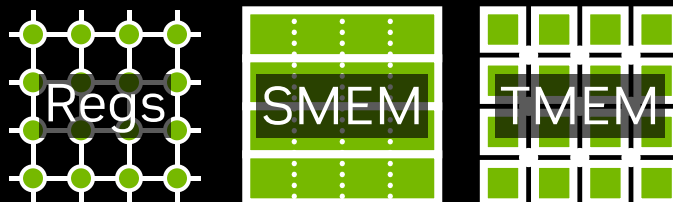
Asynchronous Block Copy



Tile memory layout?

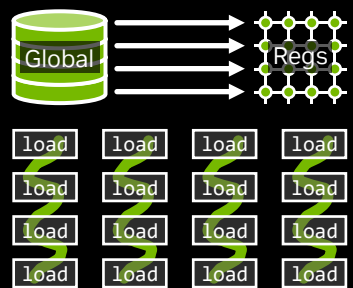


Tile storage?

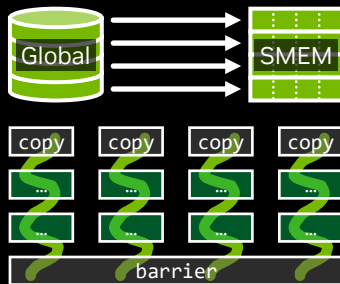


Memory movement strategy?

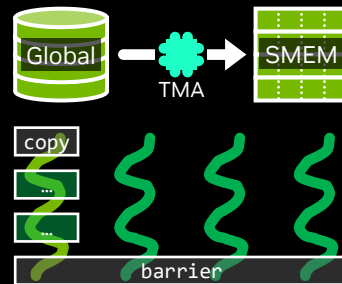
Synchronous Thread Load



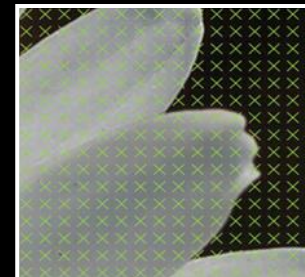
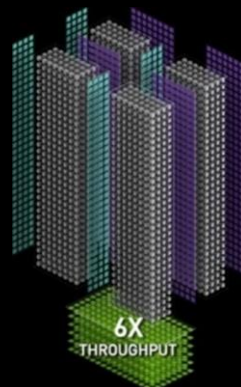
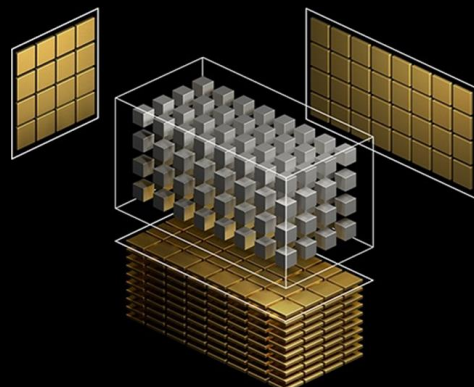
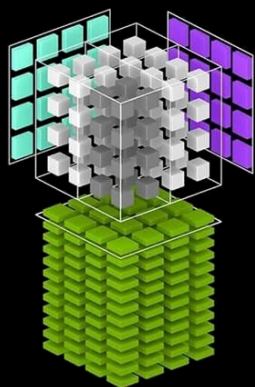
Asynchronous Thread Copy



Asynchronous Block Copy



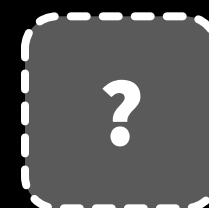
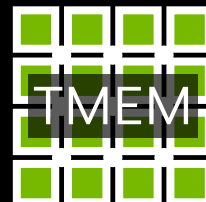
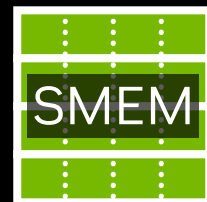
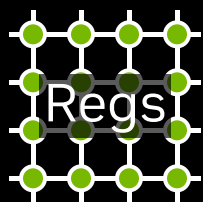
Compute strategy?



Tile memory layout?



Tile storage?

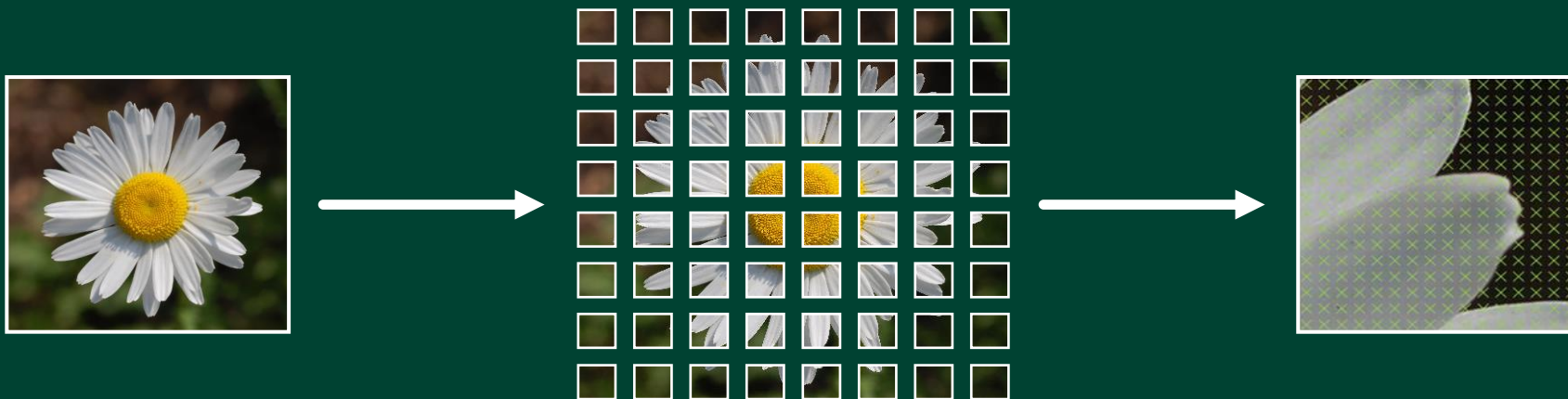


SIMT

User Responsibility

Split work into blocks
Divide data into tiles

Map blocks & tiles
onto threads



Memory movement
strategy?

Compute
strategy?

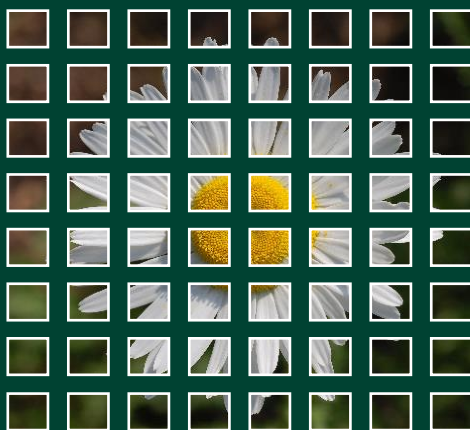
Tile memory
layout?

Tile
storage?

Tile

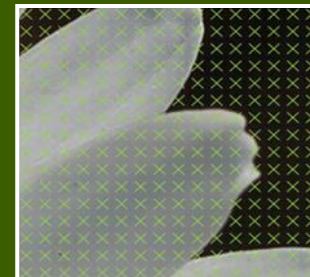
User Responsibility

Split work into blocks
Divide data into tiles



System Responsibility

Map blocks & tiles
onto threads



Memory movement
strategy?

Compute
strategy?

Tile memory
layout?

Tile
storage?

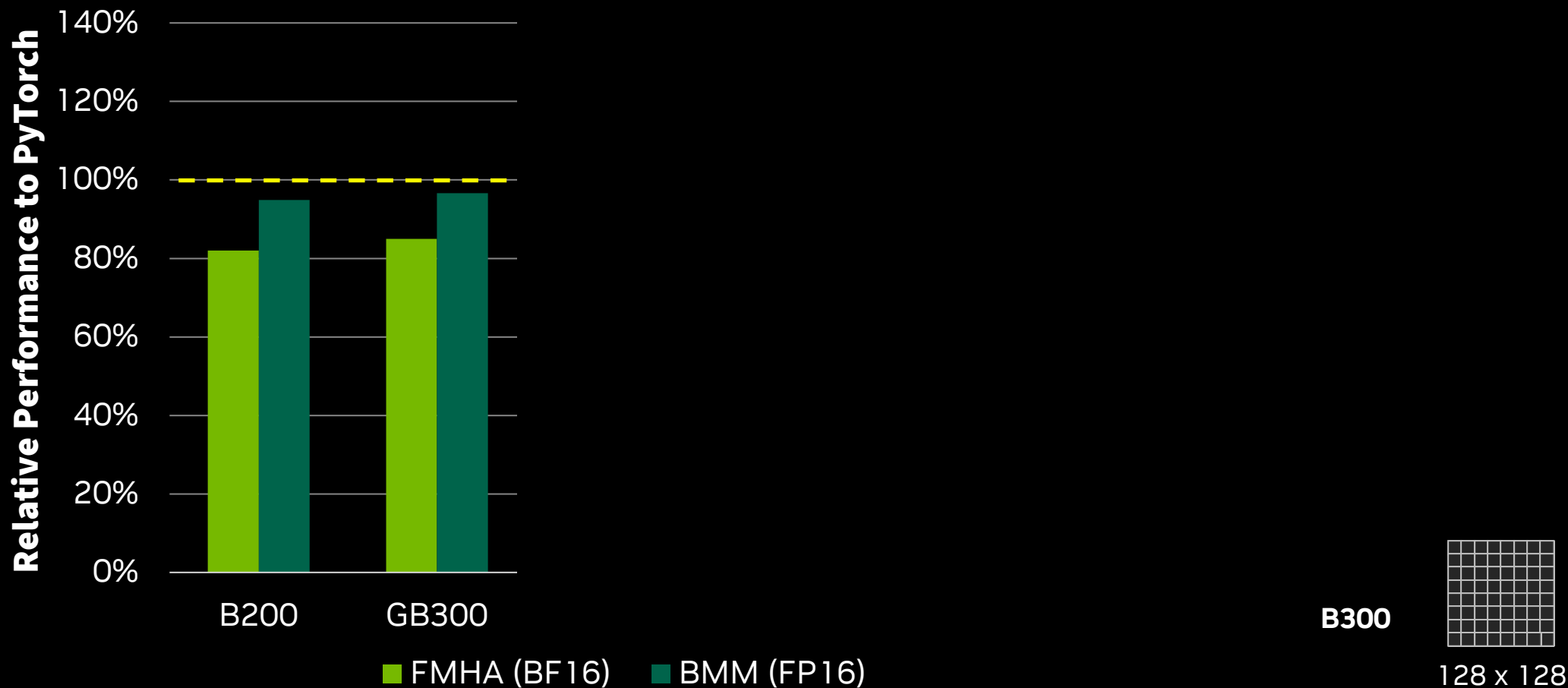
Zero Code Change Performance Portability with CUDA Tile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



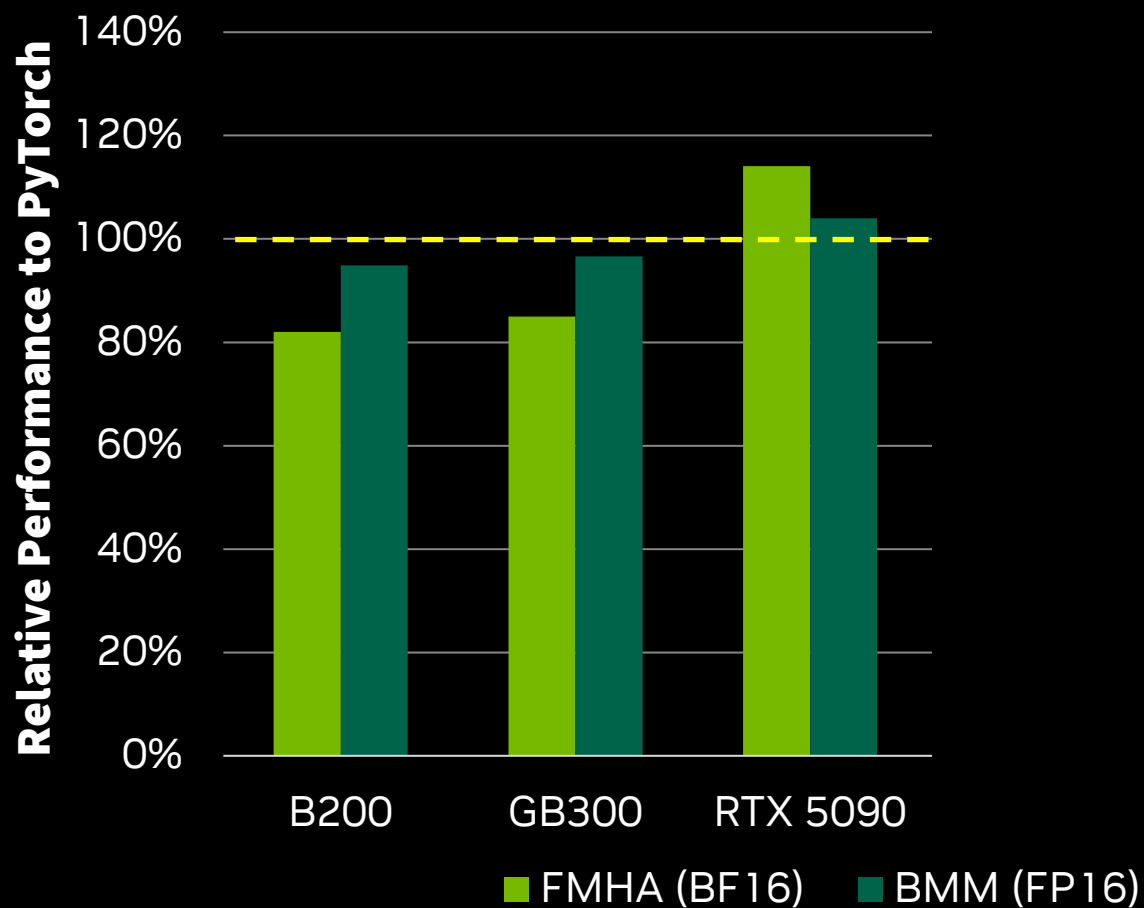
Zero Code Change Performance Portability with cuTile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



Zero Code Change Performance Portability with cuTile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



RTX
5090

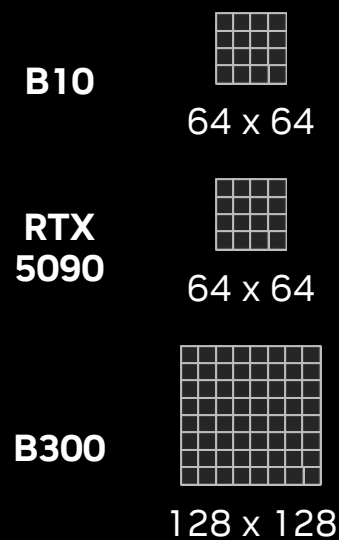
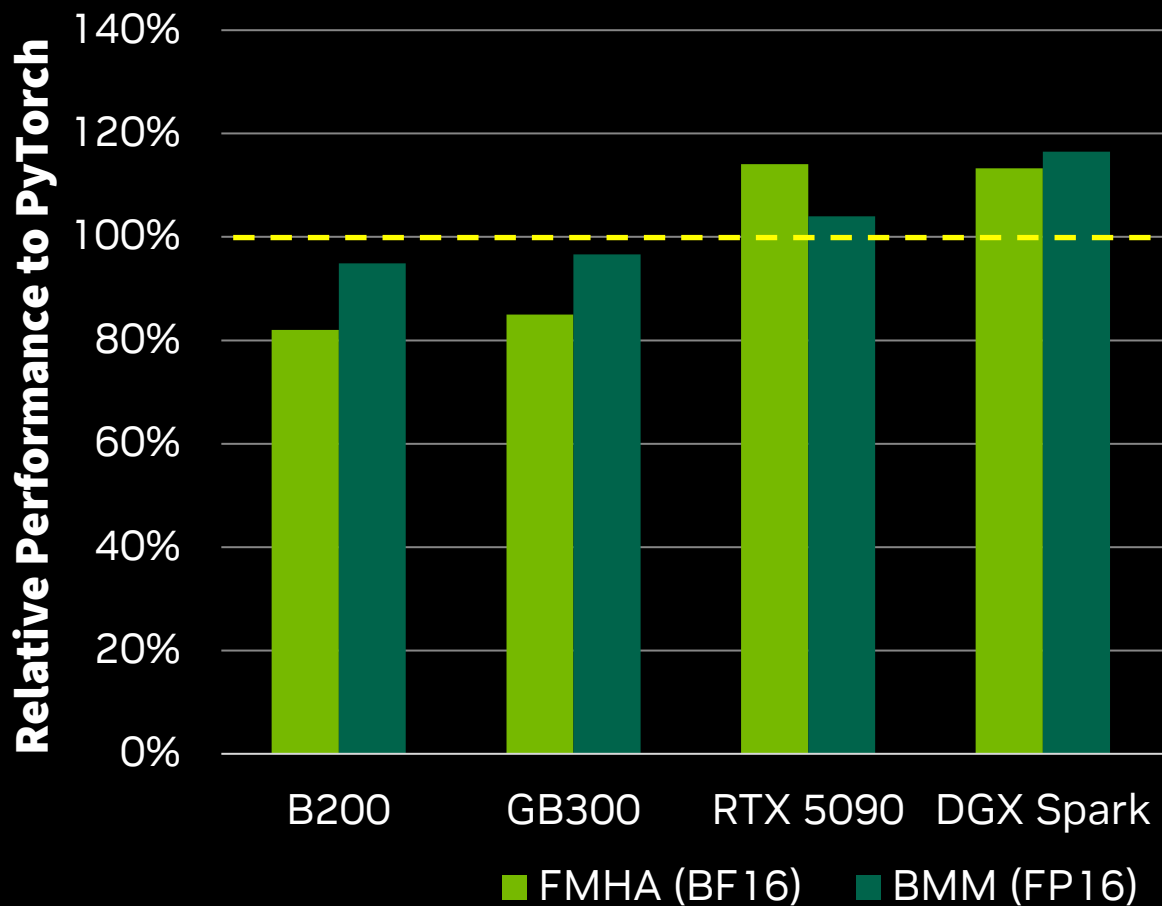
64 x 64

B300

128 x 128

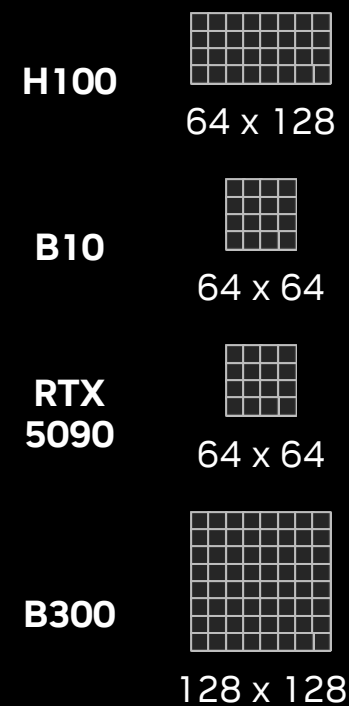
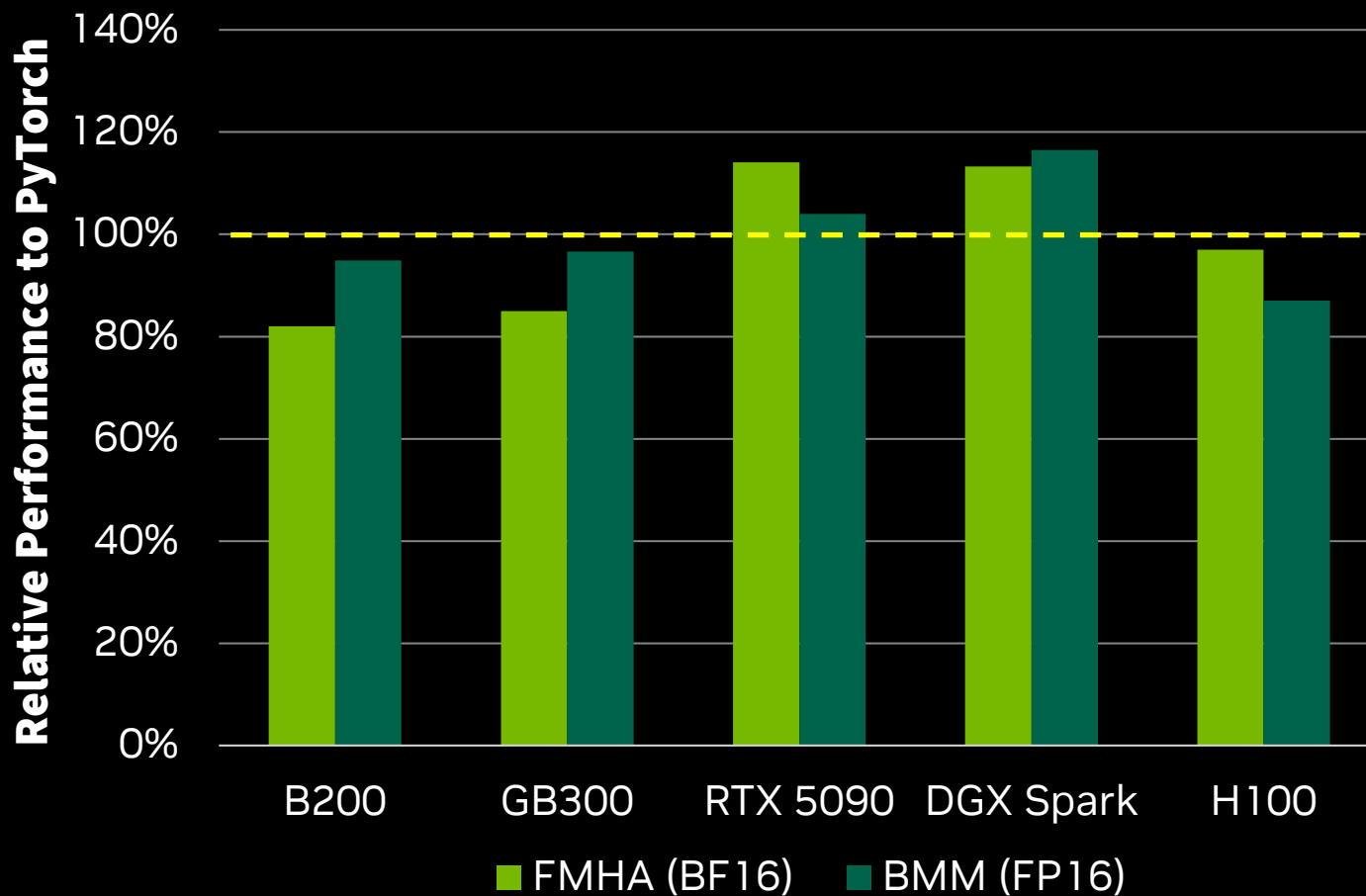
Zero Code Change Performance Portability with cuTile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



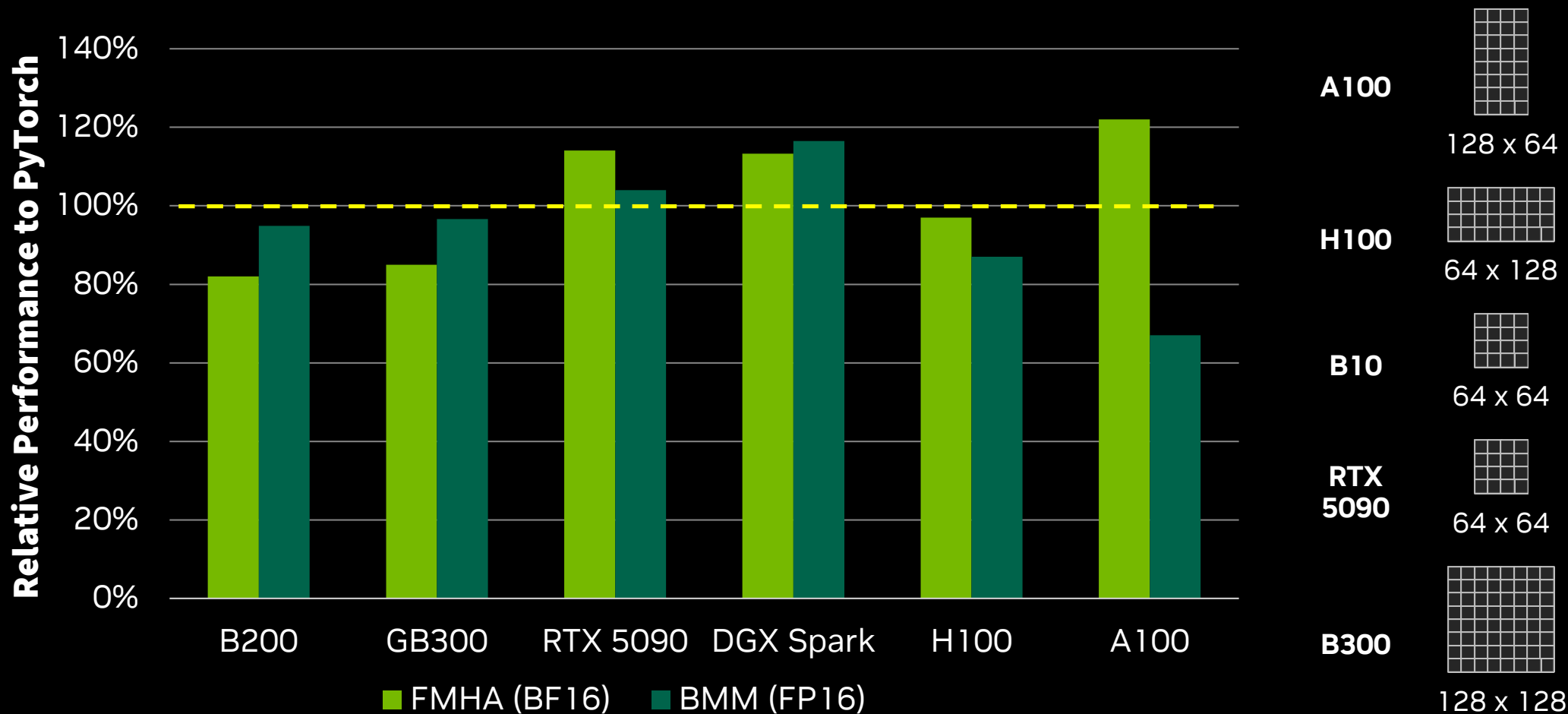
Zero Code Change Performance Portability with cuTile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



Zero Code Change Performance Portability with cuTile

Relative to PyTorch, FlashInfer kernels, average of 17 sizes per test, with autotuning



If you know NumPy, you know cuTile

NumPy

```
import numpy as np

def softmax(I, O):

    max = np.max(I, axis=1, keepdims=True)
    num = np.exp(I - max)
    den = np.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    O[:] = smax
```

cuTile Python

```
import cuda.tile as ct

@ct.kernel
def softmax(I, O, r: ct.CInt, c: ct.CInt):
    T = ct.load(I, (ct.bid(0), 0), (r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, (ct.bid(0), 0), smax)
```


Tile programming is array-based

```
import cuda.tile as ct

@ct.kernel
def softmax(I: ct.Array, O: ct.Array,
            r: ct.Constant[int], c: ct.Constant[int]):
    T = ct.load(I, index=(ct.bid(0), 0), shape=(r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, index=(ct.bid(0), 0), tile=smax)
```

Tile programming is array-based

```
import cuda.tile as ct

@ct.kernel
def softmax(I: ct.Array, O: ct.Array,
            r: ct.Constant[int], c: ct.Constant[int]):
    T = ct.load(I, index=(ct.bid(0), 0), shape=(r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, index=(ct.bid(0), 0), tile=smax)
```

Global arrays are
mutable and accessible
by all blocks.

Tile programming is array-based

```
import cuda.tile as ct

@ct.kernel
def softmax(I: ct.Array, O: ct.Array,
            r: ct.Constant[int], c: ct.Constant[int]):
    T = ct.load(I, index=(ct.bid(0), 0), shape=(r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, index=(ct.bid(0), 0), tile=smax)
```

Global arrays are
mutable and accessible
by all blocks.

Tile arrays are
immutable and local to
a block.

Tile arrays are immutable
and all tile operations return
copies, not references.

```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```

```
@ct.kernel
def first_to_42(I: ct.Array, O: ct.Array,
               r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    X[0] = 42

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=X)
```

Intuition

Scalar copies are fine; they won't be materialized.

```
a = 42
b = 17
c = a + b
d = c * c
e = d / 2
```

Array copies are bad; they will always lead to an allocation.

```
a = np.ndarray(...)
b = np.ndarray(...)
c = a + b
d = c * c
e = d / 2
```

Treat tiles as you would integers; they won't be materialized unnecessarily.

```
a = 42
b = 17
c = a + b
d = c * c
e = d / 2
```

```
a = ct.load(...)
b = ct.load(...)
c = a + b
d = c * c
e = d / 2
```

```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```



```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```

```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```

```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

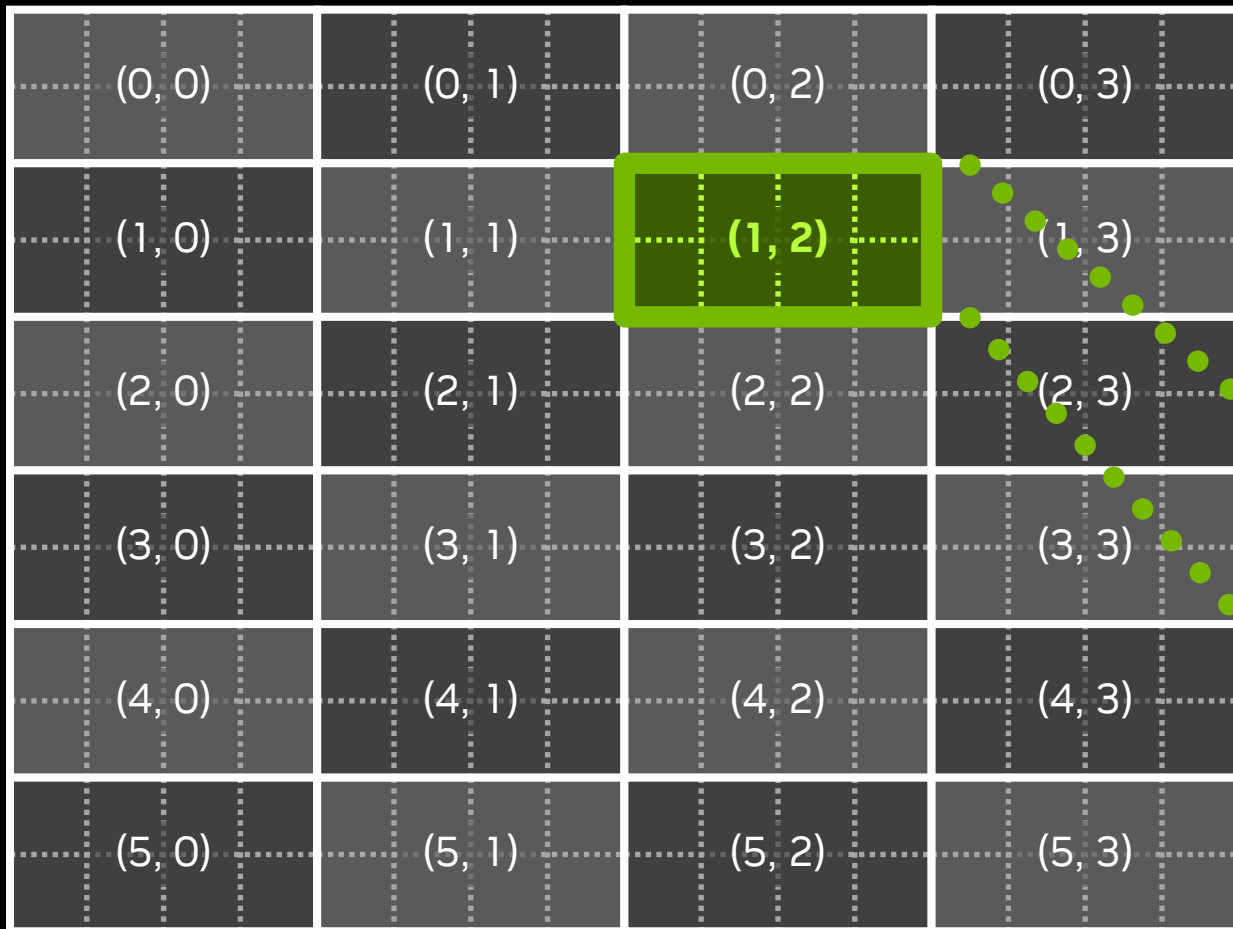
    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```

```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(X)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```



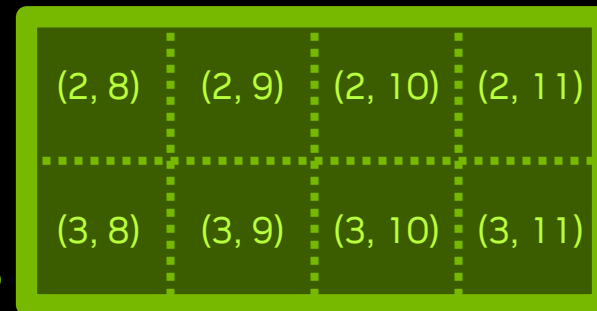
Array Shape: (12, 16)

Tile Shape: (2, 4)

Tile Grid: (6, 4)

Tile Index: (1, 2)

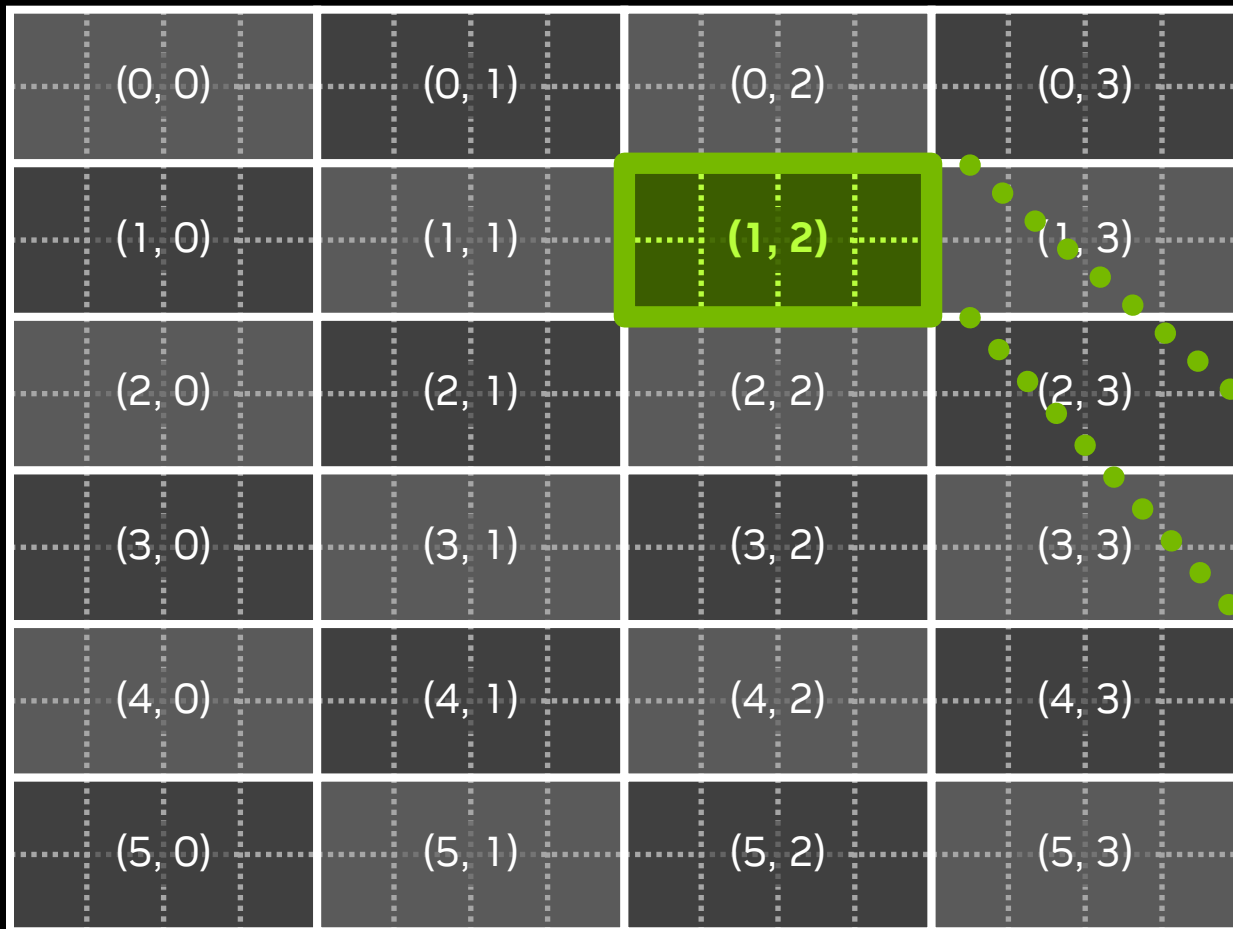
Element Indices:



```
@ct.kernel
def transpose(I: ct.Array, O: ct.Array,
              r: ct.Constant[int], c: ct.Constant[int]):
    X = ct.load(I, index=(ct.bid(0), ct.bid(1)), shape=(r, c))

    XX = ct.transpose(x)

    ct.store(O, index=(ct.bid(1), ct.bid(0)), tile=XX)
```



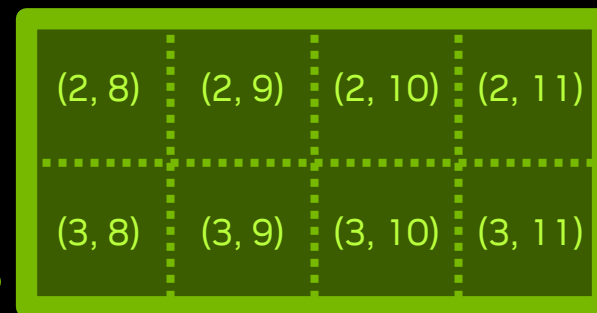
Array Shape: (12, 16)

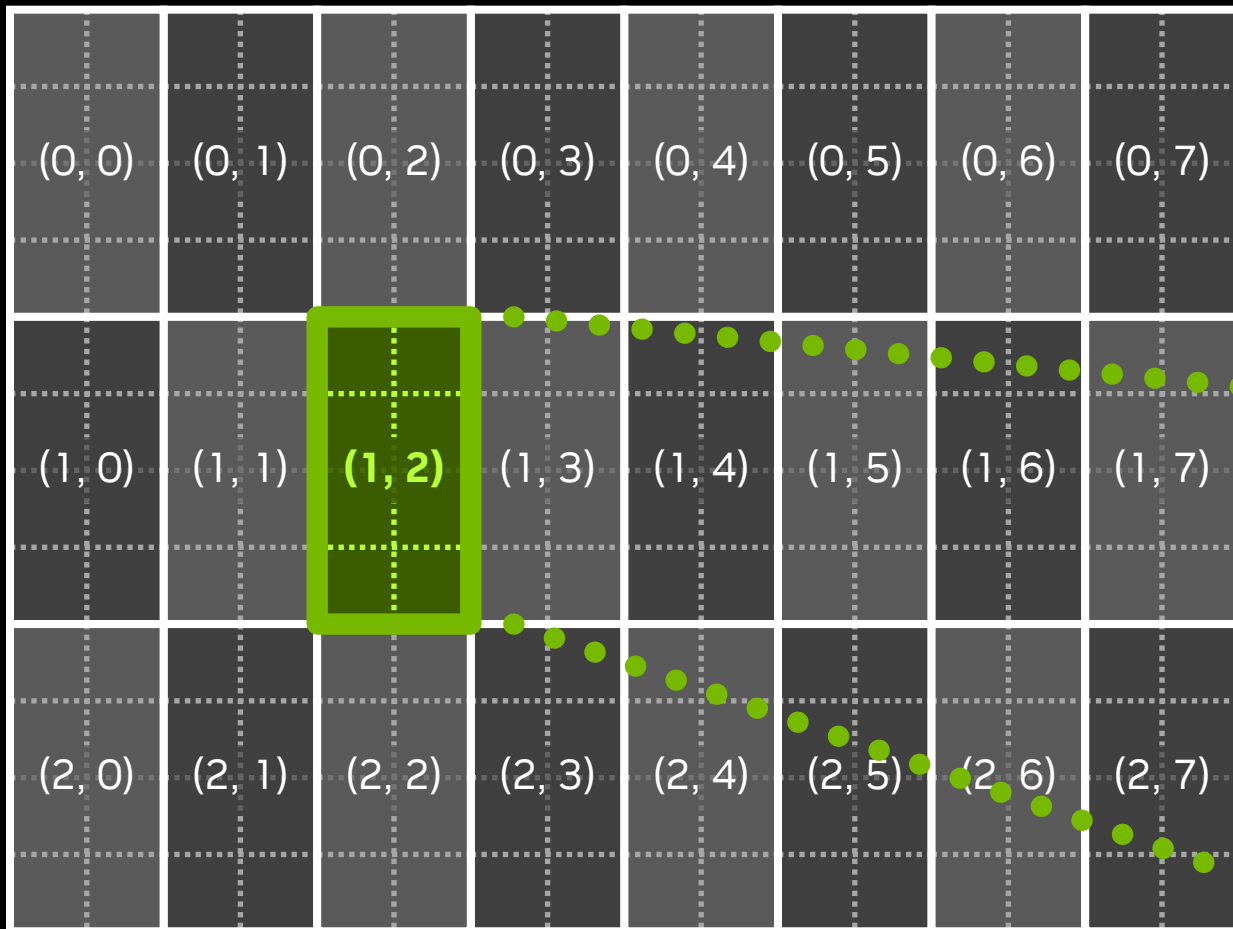
Tile Shape: (2, 4)

Tile Grid: (6, 4)

Tile Index: (1, 2)

Element Indices:





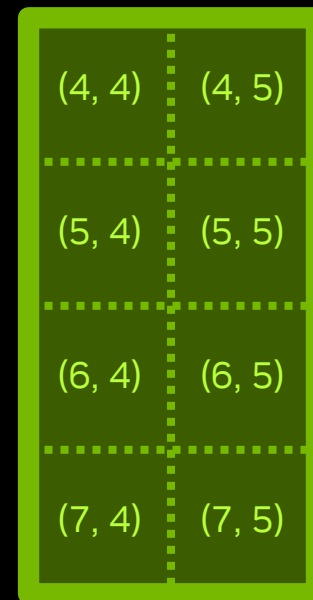
Array Shape: (12, 16)

Tile Shape: (4, 2)

Tile Grid: (3, 8)

Tile Index: (1, 2)

Element Indices:




```

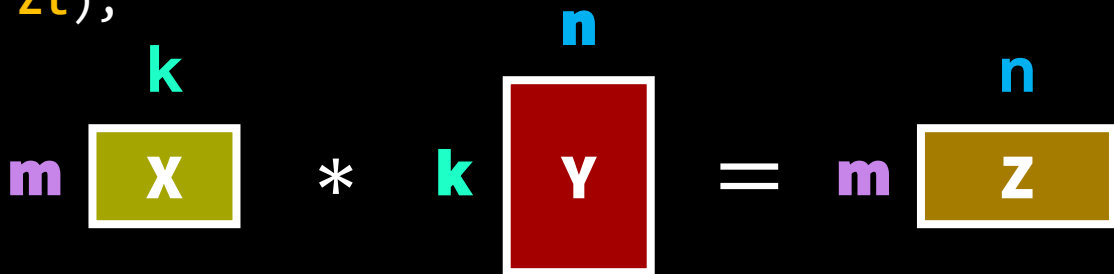
#include <cuda_tile.h>

template <int mt, int nt, int kt>
__tile__ void matmul(ct::tensor_span_like auto X,
                    ct::tensor_span_like auto Y,
                    ct::tensor_span_like auto Z) {
    using E = ct::element_t<decltype(Z)>;
    auto Zt = ct::zeros<ct::tile<E, ct::shape_v<mt, nt>>>();

    auto [i, j] = ct::swizzle2d<mt, nt>(X.extent(0), Y.extent(1));
    for (auto k : ct::irange(0, Xv.num_tiles(1))) {
        auto Xt = Xv.load(ct::shape_v<mt, kt>, i, k);
        auto Yt = Yv.load(ct::shape_v<kt, nt>, k, j);
        Zt = ct::mma(Xt, Yt, Zt);
    }

    Z.store(Zt, i, j);
}

```



```

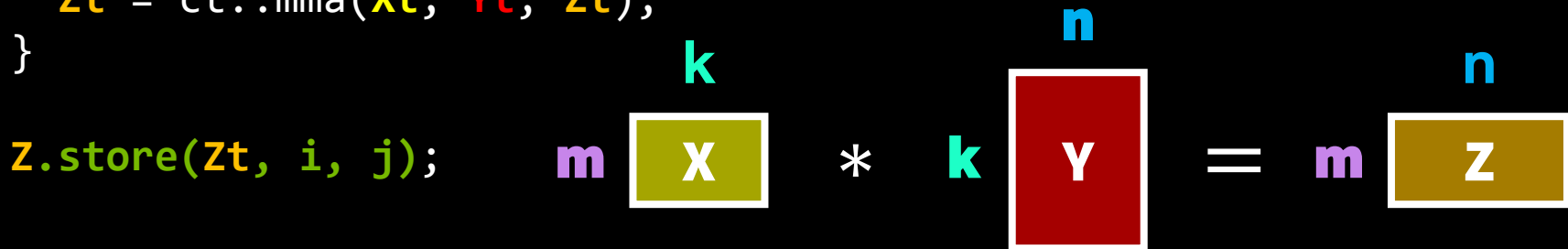
#include <cuda_tile.h>

template <int mt, int nt, int kt>
__tile__ void matmul(ct::tensor_span_like auto X,
                    ct::tensor_span_like auto Y,
                    ct::tensor_span_like auto Z) {
    using E = ct::element_t<decltype(Z)>;
    auto Zt = ct::zeros<ct::tile<E, ct::shape_v<mt, nt>>>();

    auto [i, j] = ct::swizzle2d<mt, nt>(X.extent(0), Y.extent(1));
    for (auto k : ct::irange(0, Xv.num_tiles(1))) {
        auto Xt = Xv.load(ct::shape_v<mt, kt>, i, k);
        auto Yt = Yv.load(ct::shape_v<kt, nt>, k, j);
        Zt = ct::mma(Xt, Yt, Zt);
    }

    Z.store(Zt, i, j);
}

```



`Xv.load(i, k)`

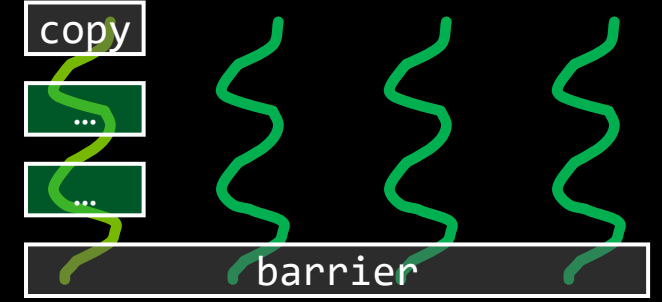
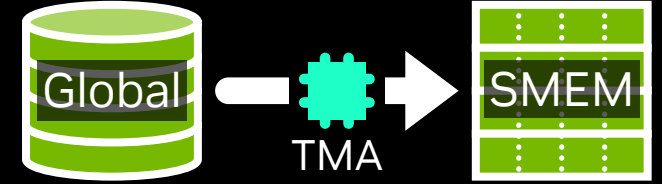
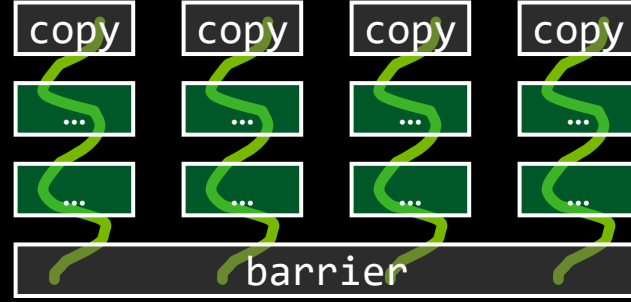
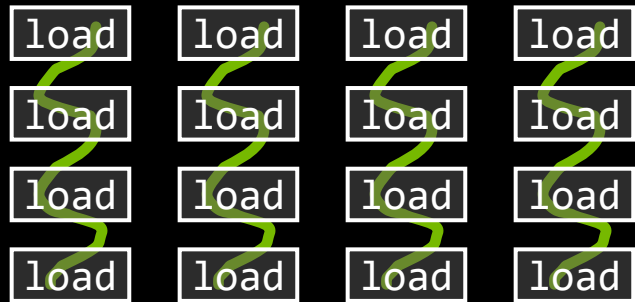
Ampere+

Hopper+
Alignment reqs
on shape/stride

Synchronous
Thread Load

Asynchronous
Thread Copy

Asynchronous
Block Copy



```

#include <cuda_tile.h>

template <int mt, int nt, int kt>
__tile__ void matmul(ct::tensor_span_like auto X,
                    ct::tensor_span_like auto Y,
                    ct::tensor_span_like auto Z) {
    using E = ct::element_t<decltype(Z)>;
    auto Zt = ct::zeros<ct::tile<E, ct::shape_v<mt, nt>>>>();

    auto [i, j] = ct::swizzle2d<mt, nt>(X.extent(0), Y.extent(1));
    for (auto k : ct::irange(0, Xv.num_tiles(1))) {
        auto Xt = Xv.load(ct::shape_v<mt, kt>, i, k);
        auto Yt = Yv.load(ct::shape_v<kt, nt>, k, j);
        Zt = ct::mma(Xt, Yt, Zt);
    }

    Z.store(Zt, i, j);
}

```

Shapes of tiles must
be known at JIT
compilation time.

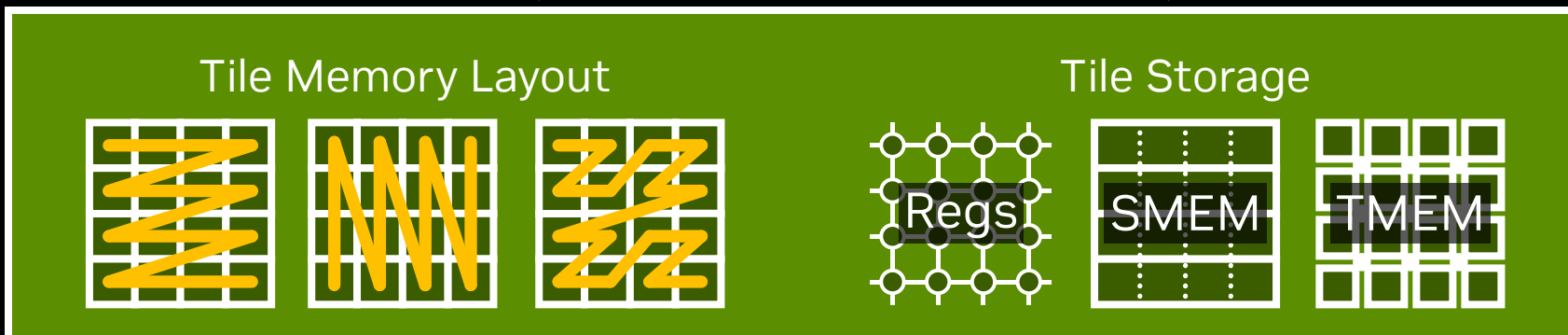
```

#include <cuda_tile.h>

template <int mt, int nt, int kt>
__tile__ void matmul(ct::tensor_span_like auto X,
                    ct::tensor_span_like auto Y,
                    ct::tensor_span_like auto Z) {
    using E = ct::element_t<decltype(Z)>;
    auto Zt = ct::zeros<ct::tile<E, ct::shape_v<mt, nt>>>();

    auto [i, j] = ct::swizzle2d<mt, nt>(X.extent(0), Y.extent(1));
    for (auto k : ct::irange(0, Xv.num_tiles(1))) {
        auto Xt = Xv.load(ct::shape_v<mt, kt>, i, k);
    }
}

```



```

template <int ROWS = 32, int COLS = 16, int OCC = 8> [[cutile::hint(0, occupancy=OCC)]]
__tile_global__ void kernel_spmv_sell( int num_rows, const int* __restrict__
sell_col_indices,
    const float* __restrict__ sell_values, const int* __restrict__ slice_offsets,
    const int* __restrict__ slice_widths, const int* __restrict__ row_perm,
    const float* __restrict__ vector_x, float* __restrict__ vector_y
) {
    // ... Setup code ...
    Tile2D acc = ct::zeros<Tile2D>();
    int full_width = width / COLS * COLS;
    ColTile cols_2d = ct::load_masked(
        sell_col_indices + offset + col_base_2d * ROWS + row_2d, row_valid_2d, 0);
    Tile2D vals_2d = ct::load_masked(
        sell_values + offset + col_base_2d * ROWS + row_2d, row_valid_2d, 0.0f);

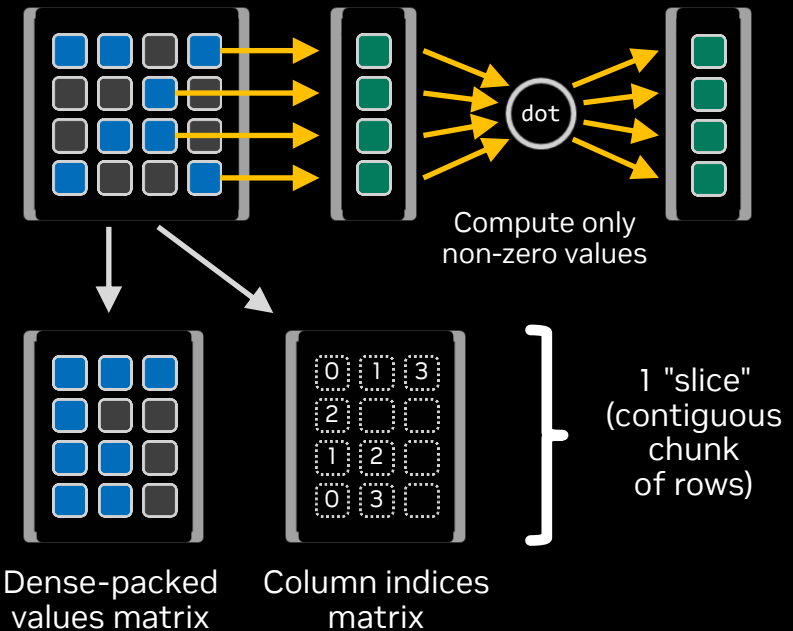
    for (int k = 0; k < full_width; k += COLS) {
        auto x_2d = ct::load_masked(vector_x + cols_2d, row_valid_2d, 0.0f);
        acc = acc + vals_2d * x_2d;
        int k_next = k + COLS;
        if (k_next < full_width) {
            auto sell_idx_next = offset + (k_next + col_base_2d) * ROWS + row_2d;
            cols_2d = ct::load_masked(sell_col_indices + sell_idx_next, row_valid_2d, 0);
            vals_2d = ct::load_masked(sell_values + sell_idx_next, row_valid_2d, 0.0f);
        }
    }

    if (full_width < width) {
        auto col_offsets_2d = full_width + col_base_2d;
        auto valid = row_valid_2d & (col_offsets_2d < width);
        auto sell_idx = offset + col_offsets_2d * ROWS + row_2d;
        auto cols_2d = ct::load_masked(sell_col_indices + sell_idx, valid, 0);
        auto vals_2d = ct::load_masked(sell_values + sell_idx, valid, 0.0f);
        auto x_2d = ct::load_masked(vector_x + cols_2d, valid, 0.0f);
        acc = acc + vals_2d * x_2d;
    }

    auto row_sums = ct::sum(acc, 1_ic);
    auto row_sums_1d = ct::reshape(row_sums, ct::shape<ROWS>{});
    ct::store_masked(vector_y + actual_row, row_sums_1d, row_valid);
}

```

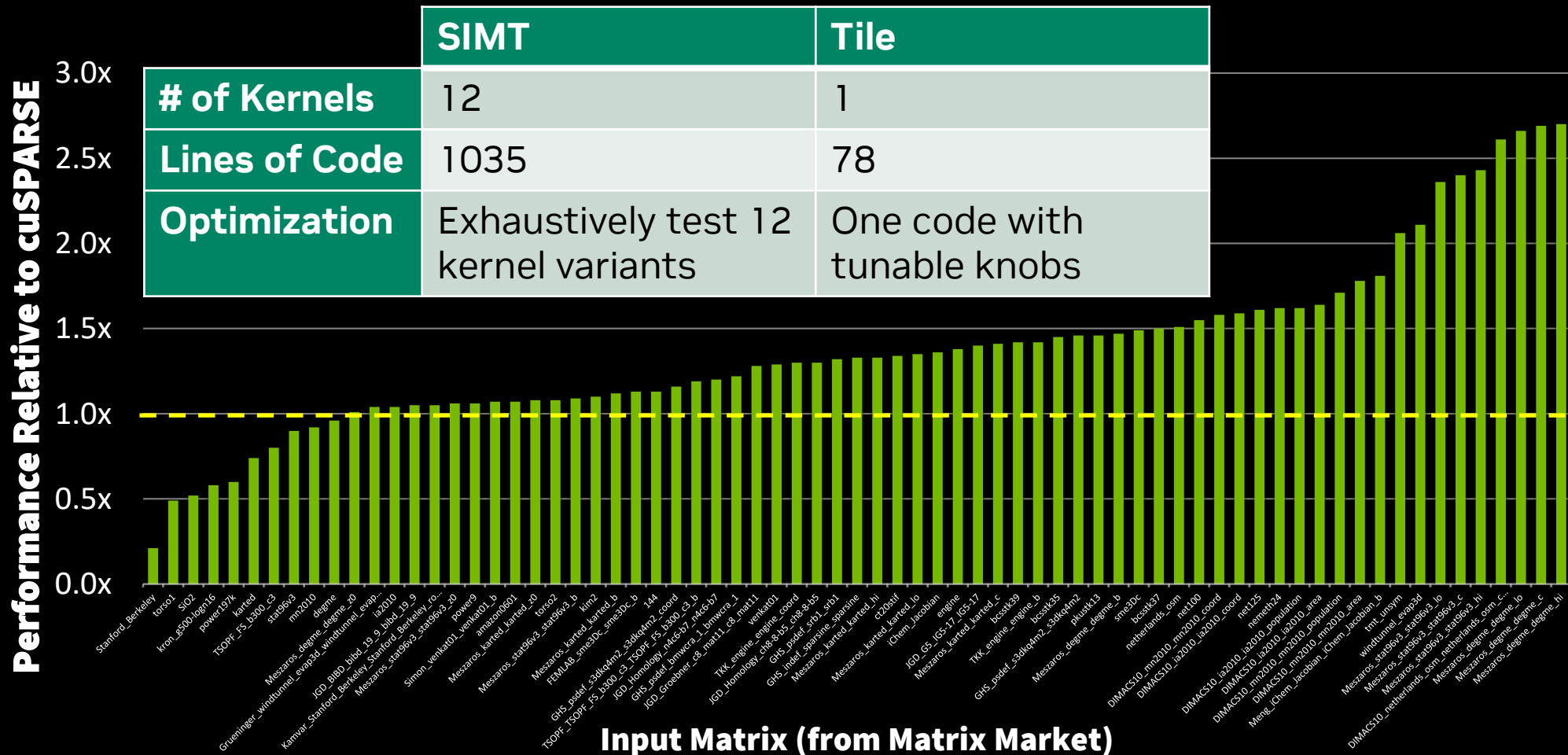
Sparse Matrix-Vector Multiply (SELLPACK)



SELLPACK (Sliced ELLPACK) uses 2-D arrays for more consistent memory access across rows.

Sparse Matrix Vector Multiplication with CUDA Tile

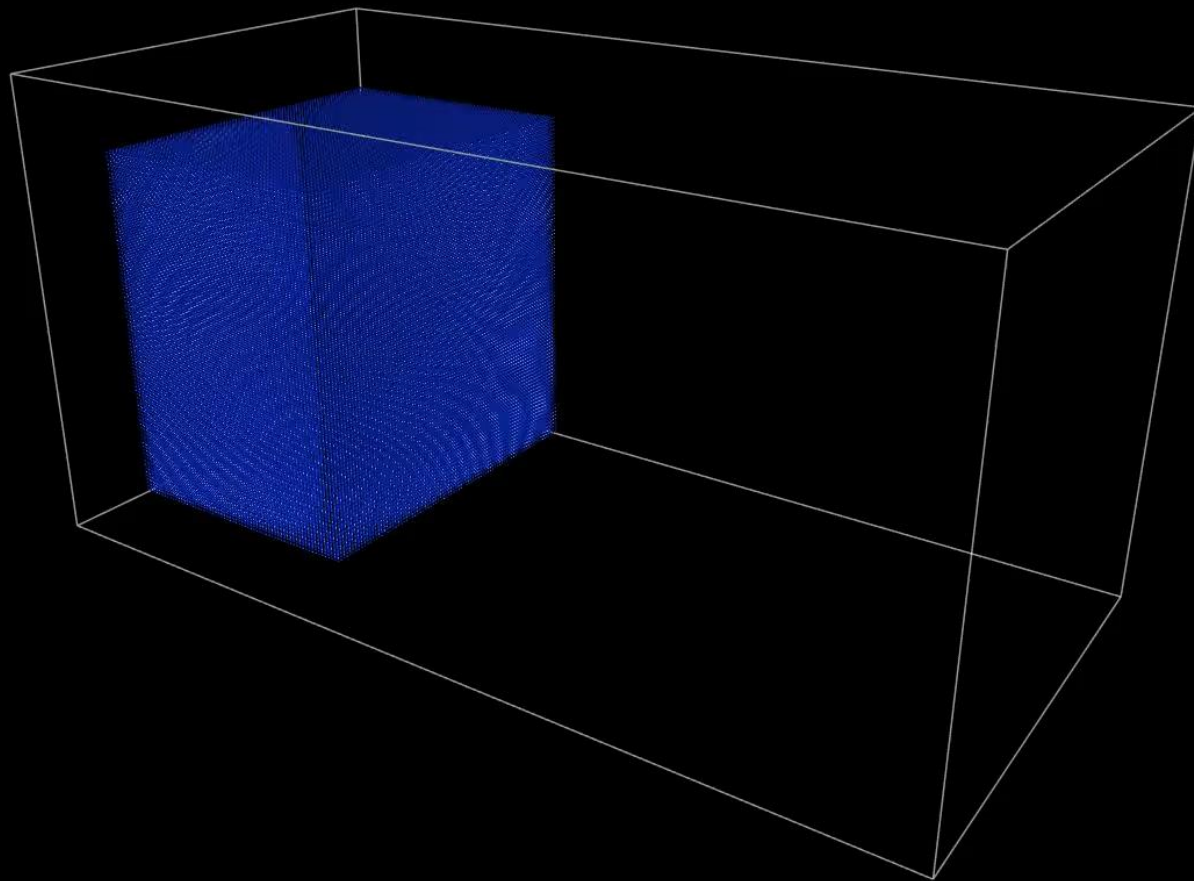
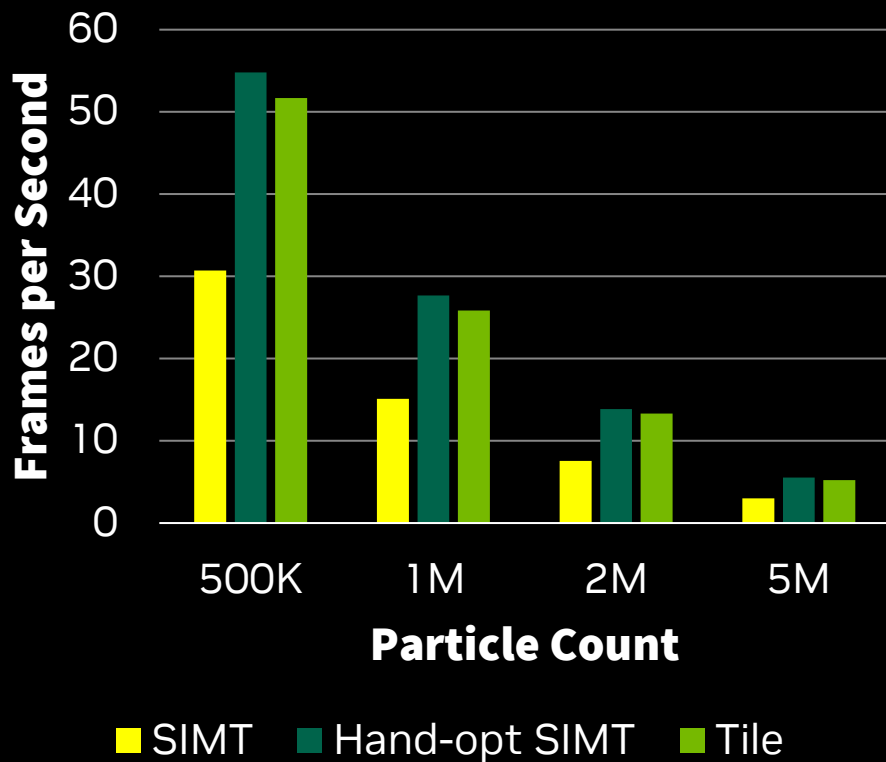
Relative to cuSPARSE bandwidth, Sliced ELLPACK, B200



Smooth Particle Hydrodynamics with CUDA Tile

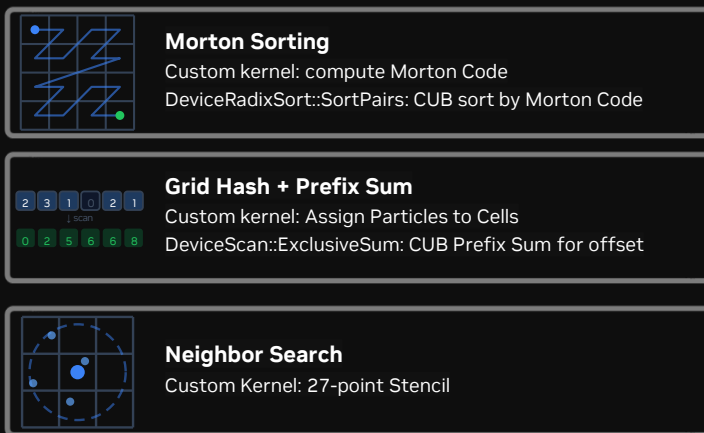
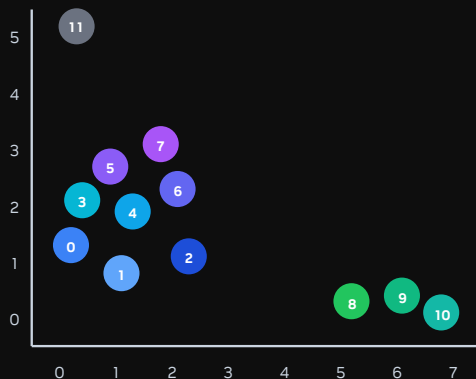
Real-Time Performance

RTX 5080



Smooth Particle Hydrodynamics with CUDA Tile

① Morton Sort · Neighbor Search · CSR Packing (SIMT)



② Density Update SpMV (TILE)



③ Force Gather & Reduce (TILE)



```
__global__ void compact_csr(  
    int n, const int* flat_nbrs,  
    const int* row_offsets, int* col_indices);
```

```
__tile_global__ void tile_density_csr_3d(  
    int num_rows, const int* row_offsets,  
    const int* col_indices,  
    const float* px,  
    const float* py,  
    const float* pz);
```

```
// ... Neighbor computation...
```

```
// Build row offsets.  
cub::DeviceScan::ExclusiveSum(  
    nbr_counts, row_offsets);
```

```
// Pack into col_indices.  
compact_csr<<<blocks, 256>>>(...);
```

```
// Density computation.  
tile_density_csr_3d<<<density_grid, 1>>>(...);
```

```
// ... Force & position update ...
```

Today,
write & use
Tile & SIMT
kernels in the
same source file.

Tile Kernel

SIMT Kernel

```
__global__ void compact_csr(
    int n, const int* flat_nbrs,
    const int* row_offsets, int* col_indices);
```

```
__tile_global__ void tile_density_csr_3d(
    int num_rows, const int* row_offsets,
    const int* col_indices,
    const float* px,
    const float* py,
    const float* pz);
```

```
// ... Neighbor computation...
```

```
// Build row offsets.
```

```
cub::DeviceScan::ExclusiveSum(
    nbr_counts, row_offsets);
```

```
// Pack into col_indices.
```

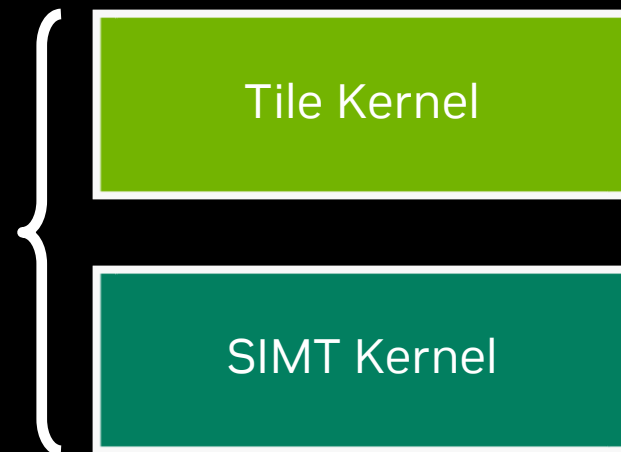
```
compact_csr<<<blocks, 256>>>(...);
```

```
// Density computation.
```

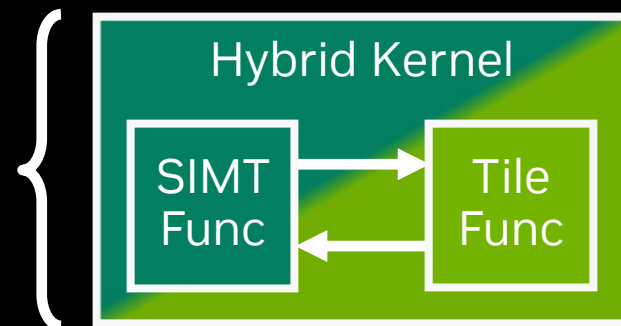
```
tile_density_csr_3d<<<density_grid, 1>>>(...);
```

```
// ... Force & position update ...
```

Today,
write & use
Tile & SIMT
kernels in the
same source file.



In the future,
write kernels
that use both
Tile & SIMT
functions



```

@ct.kernel
def spectral_histogram(y_in: ct.Array,          # [B, M, N]
                      sorted_counts_out: ct.Array, # [B, BINS]
                      sorted_bins_out: ct.Array,  # [B, BINS]
                      scan_out: ct.Array,         # [B, BINS]
                      M: ct.Constant[int], N: ct.Constant[int],
                      BINS: ct.Constant[int],
                      NUM_ITEMS: ct.Constant[int]):
    y = ct.load(y_in, (ct.bid(0), 0, 0), (1, M, N)).reshape(y, (M, N))

    spec = nvmath.tile.fft(y)
    power = spec.real * spec.real + spec.imag * spec.imag

    lv = ct.log(power + 1.0)
    scaled = (lv - ct.min(lv)) / (ct.max(lv) - ct.min(lv) + EPS)
    scaled = ct.max(ct.min(scaled, 1.0), 0.0)

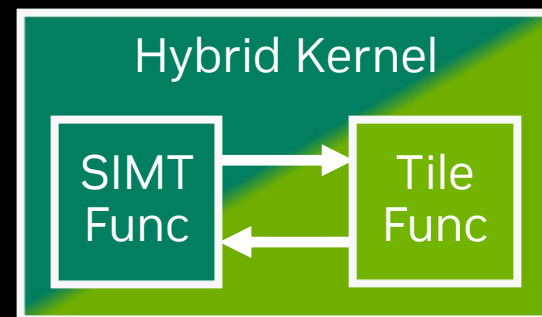
    q = ct.floor(scaled * (BINS - 1))
    keys = ct.reshape(ct.astype(q, ct.uint8), (NUM_ITEMS,))

    counts = cuda.tile.coop.histogram(keys, bins=BINS)
    sorted_counts, sorted_bins = cuda.coop.tile.radix_sort_pairs(
        counts, ct.arange(BINS, dtype=ct.uint32, end_bit=32)
    )
    scan_counts = cuda.coop.tile.exclusive_sum(counts)

    ct.store(sorted_counts_out, (ct.bid(0),), sorted_counts)
    ct.store(sorted_bins_out, (ct.bid(0),), sorted_bins)
    ct.store(scan_out, (ct.bid(0),), scan_counts)

```

In the future,
write kernels that use
both Tile & SIMT
functions.



```

@ct.kernel
def spectral_histogram(y_in: ct.Array,          # [B, M, N]
                      sorted_counts_out: ct.Array, # [B, BINS]
                      sorted_bins_out: ct.Array,   # [B, BINS]
                      scan_out: ct.Array,         # [B, BINS]
                      M: ct.Constant[int], N: ct.Constant[int],
                      BINS: ct.Constant[int],
                      NUM_ITEMS: ct.Constant[int]):
    y = ct.load(y_in, (ct.bid(0), 0, 0), (1, M, N)).reshape(y, (M, N))

    spec = nvmath.tile.fft(y)
    power = spec.real * spec.real + spec.imag * spec.imag

    lv = ct.log(power + 1.0)
    scaled = (lv - ct.min(lv)) / (ct.max(lv) - ct.min(lv) + EPS)
    scaled = ct.max(ct.min(scaled, 1.0), 0.0)

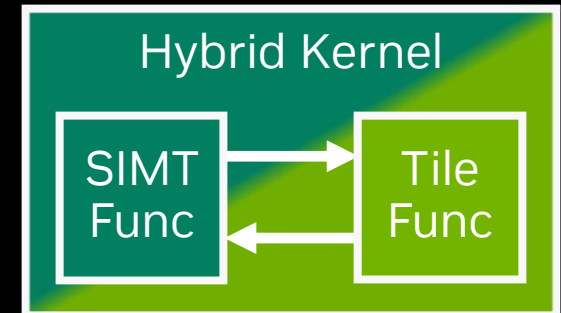
    q = ct.floor(scaled * (BINS - 1))
    keys = ct.reshape(ct.astype(q, ct.uint8), (NUM_ITEMS,))

    counts = cuda.tile.coop.histogram(keys, bins=BINS)
    sorted_counts, sorted_bins = cuda.coop.tile.radix_sort_pairs(
        counts, ct.arange(BINS, dtype=ct.uint32, end_bit=32)
    )
    scan_counts = cuda.coop.tile.exclusive_sum(counts)

    ct.store(sorted_counts_out, (ct.bid(0),), sorted_counts)
    ct.store(sorted_bins_out, (ct.bid(0),), sorted_bins)
    ct.store(scan_out, (ct.bid(0),), scan_counts)

```

In the future,
write kernels that use
both Tile & SIMT
functions.



```

@ct.kernel
def spectral_histogram(y_in: ct.Array,          # [B, M, N]
                      sorted_counts_out: ct.Array, # [B, BINS]
                      sorted_bins_out: ct.Array,  # [B, BINS]
                      scan_out: ct.Array,         # [B, BINS]
                      M: ct.Constant[int], N: ct.Constant[int],
                      BINS: ct.Constant[int],
                      NUM_ITEMS: ct.Constant[int]):
    y = ct.load(y_in, (ct.bid(0), 0, 0), (1, M, N)).reshape(y, (M, N))

    spec = nvmath.tile.fft(y)
    power = spec.real * spec.real + spec.imag * spec.imag

    lv = ct.log(power + 1.0)
    scaled = (lv - ct.min(lv)) / (ct.max(lv) - ct.min(lv) + EPS)
    scaled = ct.max(ct.min(scaled, 1.0), 0.0)

    q = ct.floor(scaled * (BINS - 1))
    keys = ct.reshape(ct.astype(q, ct.uint8), (NUM_ITEMS,))

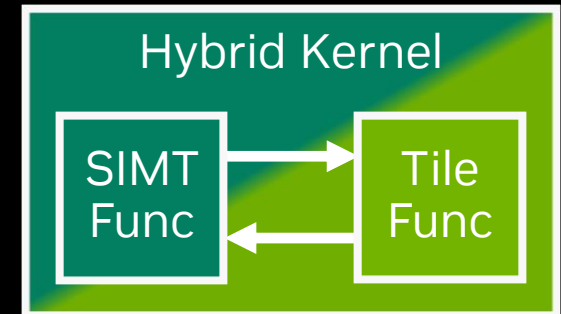
    counts = cuda.tile.coop.histogram(keys, bins=BINS)
    sorted_counts, sorted_bins = cuda.coop.tile.radix_sort_pairs(
        counts, ct.arange(BINS, dtype=ct.uint32, end_bit=32)
    )
    scan_counts = cuda.coop.tile.exclusive_sum(counts)

    ct.store(sorted_counts_out, (ct.bid(0),), sorted_counts)
    ct.store(sorted_bins_out, (ct.bid(0),), sorted_bins)
    ct.store(scan_out, (ct.bid(0),), scan_counts)

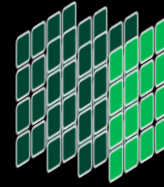
```

In the future,
write kernels that use
both Tile & SIMT
functions.

Worst case overhead for
interop calls is ~4%.



```
@ct.kernel
def fft_kernel_y(input_data: ct.Array, output_data: ct.Array):
    x, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, 0, z, 0), shape=(1, FFT_SIZE, FFTS_PER_BLOCK, 2))
    fft_result = nvmath.tile.fft_y(data)
    ct.store(output_data, (x, 0, z, 0), fft_result)
```



XZ FFT in the Y plane

```
@ct.kernel
def fft_kernel_z(input_data: ct.Array, output_data: ct.Array):
    y, x = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, y, 0, 0), shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
    fft_result = nvmath.tile.fft_z(data)
```



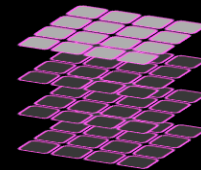
XY FFT in the Z plane

```
my_pe = nvshmem.tile.my_pe()

dist_FFT_SIZE = FFT_SIZE // num_pes
tile_size = FFTS_PER_BLOCK * FFT_SIZE * 2
local_x = x
local_y = y * FFTS_PER_BLOCK
dst_pe = local_y // dist_FFT_SIZE
dst_y = local_y % dist_FFT_SIZE
dst_x = my_pe * dist_FFT_SIZE + local_x

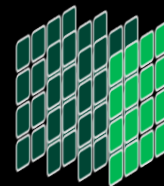
dest = ct.Array.from_raw(output_data, (dst_x, dst_y, 0, 0),
                           shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
nvshmem.tile.put(dest, fft_result, dst_pe)
```

```
@ct.kernel
def fft_kernel_x(input_data: ct.Array, output_data: ct.Array):
    y, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (0, y, z, 0), shape=(FFT_SIZE, 1, FFTS_PER_BLOCK, 2))
    fft_result = nvmath.tile.fft_x(data)
    ct.store(output_data, (0, y, z, 0), fft_result)
```



YZ FFT in the X plane

```
@ct.kernel
def fft_kernel_y(input_data: ct.Array, output_data: ct.Array):
    x, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, 0, z, 0), shape=(1, FFT_SIZE, FFTS_PER_BLOCK, 2))
    fft_result = nvmath.tile.fft_y(data)
    ct.store(output_data, (x, 0, z, 0), fft_result)
```



XZ FFT in the Y plane

```
@ct.kernel
def fft_kernel_z(input_data: ct.Array, output_data: ct.Array):

    y, x = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (x, y, 0, 0), shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
    fft_result = nvmath.tile.fft_z(data)
```



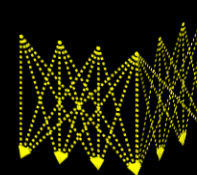
XY FFT in the Z plane

```
my_pe = nvshmem.tile.my_pe()
```

```
dist_FFT_SIZE = FFT_SIZE // num_pes
tile_size = FFTS_PER_BLOCK * FFT_SIZE * 2
local_x = x
local_y = y * FFTS_PER_BLOCK
dst_pe = local_y // dist_FFT_SIZE
dst_y = local_y % dist_FFT_SIZE
dst_x = my_pe * dist_FFT_SIZE + local_x
```

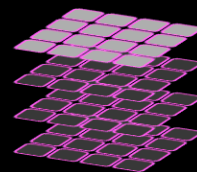
```
dest = ct.Array.from_raw(output_data, (dst_x, dst_y, 0, 0),
                          shape=(1, FFTS_PER_BLOCK, FFT_SIZE, 2))
nvshmem.tile.put(dest, fft_result, dst_pe)
```

Tile will support
multi-GPU multi-
node comms



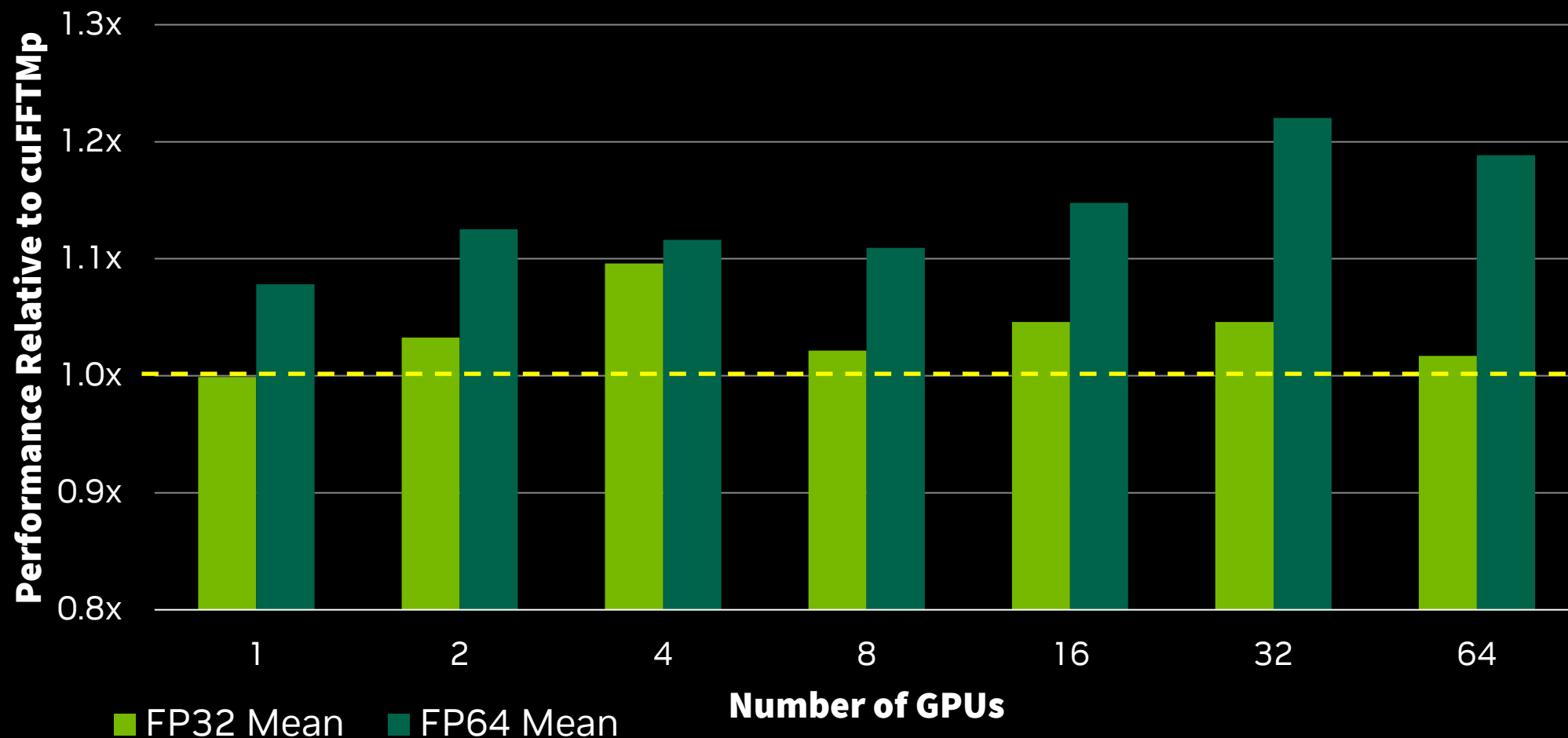
Exchange data
between nodes using
NVSHMEM

```
@ct.kernel
def fft_kernel_x(input_data: ct.Array, output_data: ct.Array):
    y, z = ct.bid(0), ct.bid(1)
    data = ct.load(input_data, (0, y, z, 0), shape=(FFT_SIZE, 1, FFTS_PER_BLOCK, 2))
    fft_result = nvmath.tile.fft_x(data)
    ct.store(output_data, (0, y, z, 0), fft_result)
```



YZ FFT in the X plane

Distributed FFT with Tile + cuFFTDx + NVSHMEM



cuTile: Tile Programming for GPUs

Array-oriented & easy to use.

Performant & portable across NVIDIA GPUs.

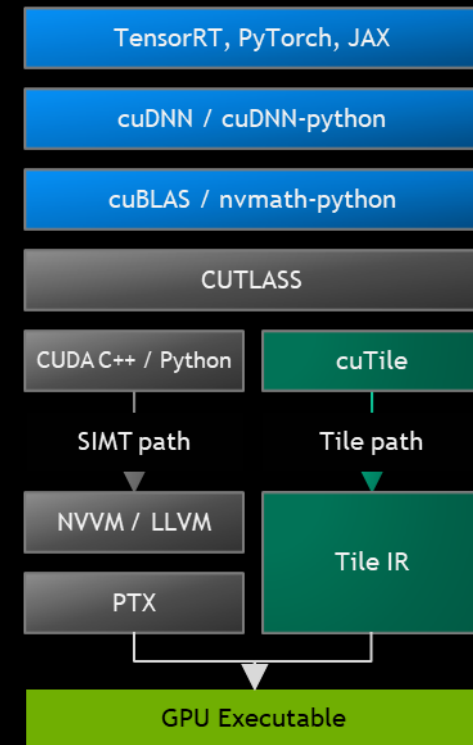
```
import cuda.tile as ct

@ct.kernel
def softmax(I, O, r: ct.CInt, c: ct.Cint):
    T = ct.load(I, (ct.bid(0), 0), (r, c))

    max = ct.max(T, axis=1, keepdims=True)
    num = ct.exp(T - max)
    den = ct.sum(num, axis=1, keepdims=True)
    smax = num / (den + 1e-6)

    ct.store(O, (ct.bid(0), 0), smax)
```

Tile IR: A New Low-Level Compiler For Tile Programs





<https://github.com/NVIDIA/TileGym>

CUTILE PROGRAMMING

Bryce Adelstein Lelbach

Principal Architect

✉ brycelelbach@gmail.com [@blelbach](https://twitter.com/blelbach)

